

Robustness of Learning-Based Vulnerability Detectors under Semantics-Preserving Code Transformations

Phat T. Tran-Truong^{a,b}, Xuan-Bach Le^{a,b}

^aFaculty of Computer Science and Engineering, Ho Chi Minh City University of Technology (HCMUT), 268 Ly Thuong Kiet Street, Dien Hong Ward, Ho Chi Minh City, Vietnam

^bVietnam National University Ho Chi Minh City, Linh Xuan Ward, Ho Chi Minh City, Vietnam

ARTICLE INFO

Keywords:

software vulnerability detection
robustness
empirical software engineering
software quality assurance
code transformation
software security
metamorphic testing

ABSTRACT

Learning-based vulnerability detectors are usually evaluated on clean benchmark functions, but software engineering workflows also require stable behavior when source code is reformatted, renamed, commented, normalized, or conservatively rewritten without changing vulnerability status. This article presents a paired metamorphic-testing benchmark for function-level vulnerability detectors. The primary experiment evaluates exact public VulBERTa and LineVul checkpoints and the official ReGVD graph implementation on the full transformed CodeXGLUE test set, covering 2,732 original functions and 21,856 transformed variants; a public GraphCodeBERT-Devign checkpoint, an AST-structural baseline, Big-Vul, DiverseVul, and Juliet/SARD provide supporting evidence. We combine thresholded flip rates with calibration-invariant score metrics and an exploratory vulnerability-dominance counterfactual pair (VDCP) probe that asks whether vulnerable pre-repair code receives higher risk than its patched counterpart. VulBERTa provides the clearest full-test signal: it flips 7.01% of transformed predictions, drops from 0.577 clean F1 to 0.555 variant F1, and adds 58.3 true-positive-to-false-negative flips per 1,000 vulnerable function-variant pairs. Calibration-invariant metrics reveal hidden instability in operating-point-confounded detectors: LineVul has only 1.22% decision flips but TIFR 9.76%, while validation-calibrated ReGVD has 0.81% decision flips but TIFR 7.23%. The VDCP probe shows that transformation invariance and vulnerability-fix ordering are distinct properties, with exact public detectors often failing to rank vulnerable code above patched code. A two-annotator audit of a random sample finds no invalid transformed examples, with a 95% upper bound of 0.43% on the invalid rate. Overall, robustness claims for vulnerability detectors must be interpreted jointly with calibration, class imbalance, predictive utility, and deployment role.

1. Introduction

Learning-based vulnerability detectors are increasingly used to support software security review, continuous integration pipelines, vulnerability triage, and security regression testing. These tools promise to help developers identify vulnerable code earlier in the software lifecycle and to reduce the manual effort required to inspect large code bases [1, 2, 3]. As a result, learning-based detectors are often discussed not only as research models, but also as components that could be embedded in software engineering workflows such as code review, continuous integration, and security regression testing. For such workflows, a detector must be more than accurate on a held-out benchmark. It must also behave predictably when the same program is written in an equivalent but syntactically different form. This makes detector robustness a systems-and-software quality concern: a model that is intended to gate, prioritize, or regress-test software changes must be validated against the kinds of behavior-preserving edits that occur in real development environments.

Most empirical evaluations of vulnerability detectors still emphasize clean benchmark performance on public datasets such as CodeXGLUE/Devign, Big-Vul, and DiverseVul [4, 1, 5, 6]. Reported metrics such as accuracy, precision, recall,

F1 score, AUC, PR-AUC, balanced accuracy, and MCC are necessary for comparing models, but they do not directly answer a practical robustness question: *will a detector preserve its prediction when code is changed in ways that should preserve program behavior and vulnerability status?* This gap is important because modern learning-based detectors often consume source code through tokenizers, abstract syntax, program graphs, or learned representations [7, 2, 8]. A model can therefore appear competitive on clean benchmark functions while remaining unstable under ordinary maintenance edits.

The problem is not hypothetical. Production code rarely remains in the exact form in which it appears in a benchmark dataset. Developers reformat files, rename identifiers, add comments, normalize style, introduce temporary dead branches during debugging, and rewrite local control flow while preserving behavior. Tooling can also alter code automatically through formatters, linters, code generators, migrations, or refactoring engines. These edits may change the lexical or structural appearance of a function without changing what the function computes or whether it contains a vulnerability. A vulnerability detector that flags a vulnerable function before a harmless edit but misses the behaviorally equivalent version after the edit is brittle as a software quality assurance tool, even if its clean benchmark F1 score is acceptable. Empirical refactoring and repository-mining studies show that behavior-preserving edits such as

 phat@hcmut.edu.vn (P.T. Tran-Truong);

lexuanbach@hcmut.edu.vn (X. Le)

ORCID(s): 0000-0003-3199-6333 (P.T. Tran-Truong)

renaming and other refactorings occur during software maintenance [9, 10, 11]. We do not claim that the transformation frequencies in this article match field edit frequencies in real commits. Instead, the transformations are controlled probes of robustness-relevant edit types, with dead-code carriers and marker-style renaming interpreted as stress tests for token reliance rather than estimates of ordinary maintenance prevalence.

Robustness under semantics-preserving transformations is therefore a natural metamorphic property for vulnerability detection. Metamorphic testing evaluates software by applying transformations that should preserve an expected relation between inputs and outputs [12]; a substantial body of code-intelligence robustness work has shown that neural code models are sensitive to identifier renaming, semantics-preserving transformations, adversarial examples, natural attacks, and perturbations of code-generation prompts [13, 14, 15, 16, 17, 18, 19, 20, 21]. If two functions are intended to be semantically equivalent and carry the same vulnerability label, then a stable detector should produce consistent predictions or, at minimum, should show only well-characterized and statistically bounded instability. Studying this property complements, rather than replaces, standard predictive performance evaluation. Clean accuracy measures whether the detector can separate vulnerable from non-vulnerable examples under the benchmark distribution; transformation robustness measures whether that behavior remains stable under realistic source-level variation.

Listing 1 shows a real CodeXGLUE/Devign test case from the paired artifact. The variant renames a function parameter and its uses, leaving the data flow unchanged. Under the public VulBERTa checkpoint, the vulnerable-class score drops from 0.781 on the original function to 0.256 on the renamed variant, crossing the fixed decision threshold and changing a true positive into a false negative. The example is not used as a proof of semantic equivalence by itself; it is a concrete instance of the paired phenomenon quantified later by the full-test results and audit.

```

1  /* Original: VulBERTa score = 0.781, predicted vulnerable
   */
2  static int mov_read_strf(MOVContext *c, AVIOContext *pb,
   MOVAtom atom)
3  {
4      AVStream *st;
5      if (c->fc->nb_streams < 1)
6          return 0;
7      if (atom.size <= 40)
8          return 0;
9      st = c->fc->streams[c->fc->nb_streams - 1];
10     if ((uint64_t)atom.size > (1 << 30))
11         return AERROR_INVALIDDATA;
12     av_free(st->codec->extradata);
13     st->codec->extradata =
14         av_mallocz(atom.size - 40 +
15                     FF_INPUT_BUFFER_PADDING_SIZE);
16     return 0;
17 }

```

```

18 /* Variant: c renamed to rvd_id0; score = 0.256, predicted
   safe */
19 static int mov_read_strf(MOVContext *rvd_id0, AVIOContext
   *pb,
20                         MOVAtom atom)
21 {
22     AVStream *st;
23     if (rvd_id0->fc->nb_streams < 1)
24         return 0;
25     if (atom.size <= 40)
26         return 0;
27     st = rvd_id0->fc->streams[rvd_id0->fc->nb_streams - 1];
28     if ((uint64_t)atom.size > (1 << 30))
29         return AERROR_INVALIDDATA;
30     av_free(st->codec->extradata);
31     st->codec->extradata =
32         av_mallocz(atom.size - 40 +
33                     FF_INPUT_BUFFER_PADDING_SIZE);
34     return 0;
35 }

```

Listing 1: Motivating CodeXGLUE example in which a harmless identifier rename changes the public VulBERTa prediction.

This article treats robustness under semantics-preserving transformations as a metamorphic property of learning-based vulnerability detectors in software engineering workflows. Our goal is not to claim novelty for code transformations in general: adversarial and transformation-based evaluation of code models is now well established. Instead, the contribution boundary is narrower and specific to vulnerability detection. We adapt the metamorphic robustness perspective to function-level vulnerability labels, exact public vulnerability-detector checkpoints, imbalance-aware security metrics, harmful true-positive-to-false-negative flips, and a two-annotator semantic audit of transformed benchmark snippets. The study treats transformations as paired tests: each transformed variant remains linked to its original function, detector outputs are compared within each pair, and robustness is reported through flip rates, confidence intervals, paired tests, and label-specific analyses.

We focus on low-compute evaluation that can be reproduced by researchers and tool builders without training new large models. The public CodeXGLUE evidence uses full-test runs for exact public VulBERTa and LineVul checkpoints [4, 1, 3, 2] and for the official ReGVD graph implementation [8], covering 2,732 original functions and 21,856 transformed variants. VulBERTa is the primary non-degenerate full-test detector signal; LineVul and ReGVD are reported as exact operating-point diagnostics. ReGVD is reported at both its upstream fixed threshold, where it collapses, and at a validation-selected threshold, where it becomes non-collapsed but weakly calibrated. A public GraphCodeBERT-Devign checkpoint is added on a stratified 512-origin subset, and local detector-family baselines are retained only as supporting evidence. CodeXGLUE is therefore the primary empirical basis for detector-specific conclusions; Big-Vul and DiverseVul are external-validity probes used to assess

transformation applicability and imbalance-aware metric behavior beyond a single benchmark distribution [5, 6]. Because Big-Vul and DiverseVul are highly imbalanced, we report PR-AUC, MCC, and balanced accuracy in addition to F1 and avoid using their sampled local-baseline results as full detector rankings. Since we evaluate atomic transformations rather than composed edit sequences, the main flip rates should be interpreted as single-edit evidence rather than as complete coverage of realistic multi-edit maintenance. Transformation validity is handled through syntax checks, deterministic pre-audit, and a completed two-annotator human annotation protocol. We additionally include a vulnerability-semantics-aware counterfactual probe on the available Big-Vul before/after fix pairs. Unlike label-preserving transformations, this probe expects a directional relation: the vulnerable pre-repair version should receive a higher risk score than the patched version. This separates two properties that are often conflated in robustness discussion: invariance to harmless source edits and sensitivity to vulnerability-removing fixes.

LLM-based vulnerability detectors are outside the empirical scope of this study because their prompt formats, sampling controls, model versions, and cost profiles make paired full-benchmark robustness evaluation a different experimental design. We cite recent LLM-detector evasion work as related evidence, but the claims in this article are limited to public learning-based function-level detectors with reproducible inference paths.

The study yields four main observations. First, the main non-degenerate full-test detector, VulBERTa, exhibits non-trivial prediction instability even when no retraining is involved: on the full transformed CodeXGLUE test set it shows a 7.01% prediction flip rate, a clean-to-variant F1 drop from 0.577 to 0.555, and 58.3 additional missed true-positive detections per 1,000 vulnerable function-variant pairs. Second, robustness varies substantially by transformation family. Identifier renaming, safe dead-code carriers, comment insertion, and dead-branch insertion are more disruptive than layout-only changes and account for most harmful vulnerable-to-safe flips. Third, apparent stability must be interpreted together with predictive quality and calibration. Single-threshold flip rates alone can hide large score-order movement: public LineVul has only 1.22% decision flips but TIFR 9.76% and Spearman score stability 0.736, while validation-calibrated ReGVD has 0.81% decision flips but TIFR 7.23%. This motivates reporting calibration-invariant score robustness alongside thresholded flips. Fourth, the exploratory patch-pair probe shows that robustness to label-preserving transformations is not the same as vulnerability-specific score ordering. On 101 Big-Vul before/after fix pairs, exact public VulBERTa, LineVul, and validation-calibrated ReGVD achieve only 30.7–45.5% VDCP on original pairs and at most 4.0% decision-pair success, motivating vulnerability-aware counterfactual evaluation as a distinct benchmark layer.

We address the following research questions:

RQ1: How much does detector performance change under semantics-preserving code transformations?

RQ2: Which transformation families cause the largest prediction instability?

RQ3: Are vulnerable functions more fragile than non-vulnerable functions under equivalent transformations?

RQ4: Can prediction aggregation over transformed variants improve robustness without retraining detectors?

This article makes the following contributions:

1. We define a paired metamorphic-testing benchmark for vulnerability detectors, covering transformations, validity screening, confidence intervals, paired tests, and label-specific fragility analysis.
2. We provide full transformed CodeXGLUE evidence for VulBERTa, LineVul, and ReGVD, with GraphCodeBERT, AST-structural, Big-Vul, DiverseVul, and Juliet/SARD results kept as scoped supporting evidence.
3. We extend robustness reporting beyond fixed-threshold flip rates with calibration-invariant score metrics and a VDCP patch-pair probe.
4. We evaluate lightweight aggregation and consistency-training probes, clarifying where variant-based robustness engineering helps and where calibration and predictive utility remain limiting factors.

The remainder of this article is structured as follows. Section 2 reviews learning-based vulnerability detection and robustness work for code models. Section 3 defines the study design, datasets, transformations, detector configurations, audit protocol, and metrics. Section 4 describes the implementation and reproducibility artifacts. Section 5 reports the empirical results for the four research questions and the vulnerability counterfactual probe. Section 6 discusses implications and mechanisms of brittleness. Section 7 summarizes threats to validity, and Section 8 concludes the article.

2. Background and Related Work

2.1. Learning-Based Vulnerability Detection

Learning-based vulnerability detection has been studied using token-based, sequence-based, transformer-based, and graph-based representations of source code. Function-level detectors typically predict whether a function contains a vulnerability, while line-level detectors additionally localize suspicious statements. Representative systems include graph-based function-level detectors such as Devign [1] and transformer-based line-level detectors such as LineVul [2]. Recent models exploit pre-trained code representations, code property graphs, or multimodal combinations of textual and structural features. Despite this progress, many evaluations emphasize clean benchmark performance and do not explicitly measure whether predictions are invariant under harmless code edits.

2.2. Vulnerability Detection Benchmarks

Public datasets make vulnerability detection research comparable and reproducible. Common benchmarks include CodeXGLUE/Devign [4, 1], Big-Vul [5], DiverseVul [6], Juliet, D2A, and ReVeal. These datasets differ in size, labeling method, project diversity, and vulnerability coverage. In this study, we evaluate CodeXGLUE/Devign as the primary balanced benchmark and add low-compute samples from Big-Vul and DiverseVul as EVPs rather than as full cross-dataset detector-ranking evidence.

Dataset quality is itself a threat to vulnerability-detection research. Prior work has noted that vulnerability corpora can contain duplicated code, project leakage, noisy labels, and distribution shifts between curated benchmarks and operational use [22, 1, 5]. Our protocol does not remove all such threats. Instead, it keeps each robustness comparison paired within the same original function, so that the main measurement asks whether a detector changes its own decision after a semantics-preserving edit rather than whether the benchmark label is perfect in isolation.

2.3. Code Transformations and Metamorphic Testing

Metamorphic testing evaluates software by applying transformations that should preserve an expected property of the output. For vulnerability detection, a natural metamorphic relation is prediction invariance: if a transformation preserves the semantics and vulnerability status of a function, the detector prediction should remain stable. This relation is useful because it allows robustness testing even when perfect behavioral equivalence proofs are impractical for arbitrary real-world code snippets. The broader machine-learning testing literature has proposed differential testing, coverage criteria, and metamorphic input generation for deep learning systems [23, 24, 25, 26]. We build on that testing perspective, but specialize the metamorphic relation, failure taxonomy, and metrics to software-vulnerability detectors whose false negatives and false positives have different operational costs in security workflows.

However, transformation validity is a central concern. Some transformations that appear semantics-preserving in isolation may change behavior in specific language contexts. We therefore use conservative transformation rules, syntax checks, and manual audits to reduce the risk of invalid variants.

2.4. Robustness of Code Intelligence Models

Robustness of neural code models has been studied well before the most recent large-model literature. Yefet et al. show that source-code models can be attacked with semantics-preserving perturbations [13]. MHM generates adversarial source examples with identifier renaming [14], and Bielik and Vechev formulate adversarial robustness for code as a structured discrete-input problem [15]. Rabin et al. and Henkel et al. evaluate generalization and semantic robustness under program transformations [16, 17]. ALERT emphasizes naturalness-aware attacks on pre-trained code models, while

CARROT studies detection, estimation, and enhancement of robustness in deep program-processing models [18, 19]. ReCode extends transformation-style robustness evaluation to code-generation models [20]. These works establish the broad premise that code models are brittle under semantics-preserving or natural transformations.

Our novelty is therefore not that transformations can expose brittleness in code models. The narrower contribution is to instantiate this robustness question for learning-based vulnerability detection: we use vulnerability labels rather than method names or generation outputs, report security-oriented failure modes such as true-positive-to-false-negative flips, run exact public vulnerability-detector checkpoints, handle imbalanced vulnerability corpora with PR-AUC/MCC/balanced accuracy, and validate transformed variants through a two-annotator semantic audit. Table 1 makes this positioning explicit. Relative to Li et al.'s TSE study of transformer code intelligence under code transformation, CARROT, and ALERT, the scientific increment is the vulnerability-detection measurement layer: paired vulnerability-label preservation, harmful missed-vulnerability flips, exact public detector operating points, and audit-backed transformation validity for security benchmark snippets. Recent work also reports that LLM-based vulnerability detectors can be evaded by syntax- and compilation-preserving changes [27]; our study cites it as related preprint evidence but does not rely on it for the main claims, which focus on low-compute, reproducible paired evaluation for public vulnerability detectors.

We also distinguish this study from broader LLM-agent vulnerability work. Agentic systems can reproduce or exploit vulnerabilities when given rich repository context, tools, and feedback loops, but our unit of analysis is the function-level detector score under paired source transformations. Thus, LLM agents and larger proprietary detectors are outside the primary scope; the article instead establishes a low-compute protocol that can be reused when such detectors expose stable inference interfaces.

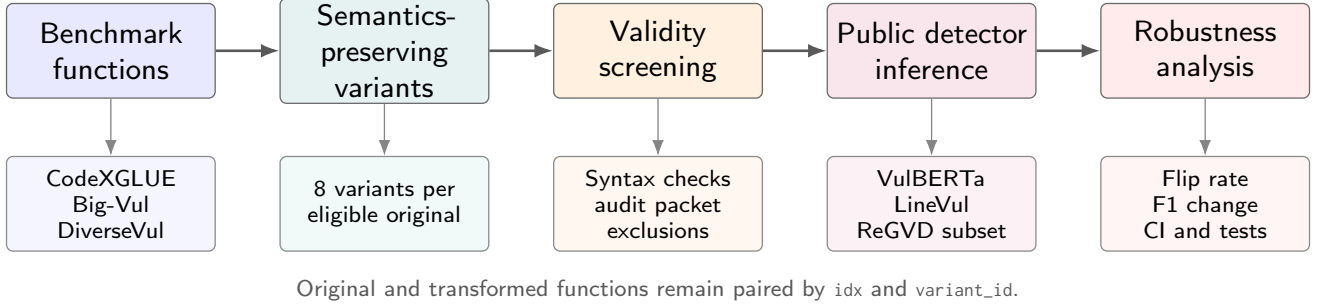
2.5. Adversarial Evasion in Security Tools

Security models have also been studied under adversarial evasion, including malware detection, phishing detection, intrusion detection, and vulnerability detection. Our study differs in emphasis. We use transformations primarily as controlled validation probes for deployed or deployable software engineering tools, not as an offensive evasion framework. In vulnerability detection specifically, prior evasion work often asks whether an attacker can bypass a model through obfuscation or adversarial rewriting [27]; our study instead asks whether routine or conservative source edits should be part of standard detector evaluation. The exact-public-checkpoint runs, paired flip taxonomy, and two-annotator semantic audit are the main additions over broader code-intelligence robustness studies.

Table 1

Positioning against closely related robustness work for code models.

Work	Main task	Transformation role	Detector evidence	Robustness metrics	Human validity audit
MHM [14]	Code classification	Adversarial identifier search	Surrogate task models	No	No
ALERT [18]	Pre-trained code models	Naturalness-aware attacks	CodeBERT-style models	No	No
CARROT [19]	Program-processing models	Robustness detection/enhancement	General code tasks	No	No
Li et al. [21]	Transformer code intelligence	Code transformations	Code-intelligence transformers	No	No
Sun et al. [27]	LLM vulnerability detection	Evasion carriers	LLM detectors	Evasion-focused	Compilation/syntax checks
This article	Function-level vulnerability detection	Paired metamorphic testing	Public VD checkpoints	TP→FN, CR, robust recall, TIFR, VDCP	Two annotators

**Figure 1:** Overview of the transformation-based robustness evaluation pipeline.

3. Study Design

3.1. Overview

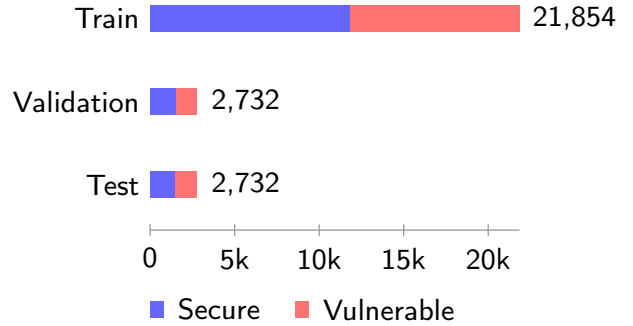
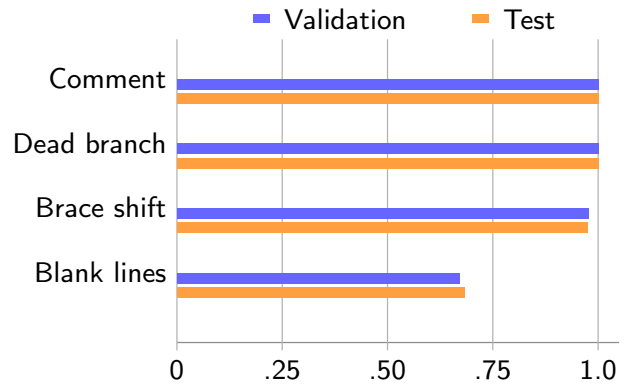
Figure 1 summarizes the study pipeline. We start from benchmark functions, generate transformed variants, validate transformation applicability, run vulnerability detectors on original and transformed functions, and compute robustness metrics.

3.2. Datasets

Table 2 summarizes the prepared datasets. The primary dataset is CodeXGLUE/Devign because it is widely used, function-level, and feasible for low-compute experiments. We additionally normalize Big-Vul and DiverseVul samples as EVPs. These samples are used to check whether the transformation pipeline remains applicable beyond CodeXGLUE, to illustrate how robustness reporting behaves under strong class imbalance, and to preserve limited external-validity context. They are not used as primary evidence for ranking detectors across datasets. Median token counts in Table 2 use the VulBERTa and LineVul tokenizers used in the exact public runs.

3.3. Transformation Taxonomy

We group transformations into six families: lexical/layout transformations, comment transformations, identifier transformations, control-flow rewrites, dead-code carrier transformations, and normalization/abstraction transformations. Table 3 summarizes the expanded transformation set. Throughout the article, “semantics-preserving” is used as shorthand for “conservatively behavior-preserving under the stated source-level preconditions.” For incomplete C/C++ benchmark snippets, this is an empirical construct supported by transformation preconditions, parse screening, audit, and limited compile anchoring rather than a formal proof of equivalence.

**Figure 2:** Class composition of CodeXGLUE/Devign splits.**Figure 3:** Transformation applicability on CodeXGLUE/Devign splits.

The completed detector results use the expanded eight-transformation suite with identifier renaming, restricted control-flow rewriting, an additional safe dead-code carrier, and return-expression normalization. These variants are generated for CodeXGLUE/Devign, Big-Vul, and DiverseVul.

Table 2

Prepared dataset statistics after normalization.

Dataset	Split	Samples	Secure	Vulnerable	Vuln. ratio	Median tokens (VulBERTa)	Median tokens (LineVul)
CodeXGLUE/Devign	Train	21,854	11,836	10,018	0.458	227	482
CodeXGLUE/Devign	Validation	2,732	1,545	1,187	0.434	224	485
CodeXGLUE/Devign	Test	2,732	1,477	1,255	0.459	236	505
Big-Vul	Test sample	3,000	2,899	101	0.034	101	175
DiverseVul	Test sample	5,000	4,743	257	0.051	133	234

Table 3

Transformation taxonomy and observed applicability on the CodeXGLUE/Devign test split.

ID	Transformation	Family	Expected semantic effect	Applicability
T1	Blank-line expansion	Lexical/layout	None	0.682
T2	Brace layout shift	Lexical/layout	None	0.974
T3	Comment insertion	Comment	None	1.000
T4	Identifier renaming	Identifier	Name-equivalent local rewrite	0.846
T5	Returning if/else inversion	Control flow	Equivalent branch order	0.003
T6	Unreachable branch insertion	Dead-code carrier	None when syntactically valid	1.000
T7	Compile-time dead-code carrier	Dead-code carrier	None when syntactically valid	1.000
T8	Return-expression parenthesization	Normalization/abstraction	Expression-equivalent normalization	0.516

The full transformed CodeXGLUE test set is used for the primary exact-checkpoint detector evidence; the Big-Vul and DiverseVul transformed samples are used for applicability and imbalance-aware reporting checks.

Identifier renaming is intentionally conservative because C/C++ name binding can interact with macros, linkage, shadowing, and preprocessor directives. The implementation protects comments, strings, character literals, and preprocessor-directive spans, never renames reserved words or existing double-underscore identifiers, and introduces a fresh marker name only when a simple local variable or parameter candidate appears at least twice outside protected spans. If no such candidate exists, or if the marker already appears in the function, the transformation is skipped and the skip reason is recorded. This is not full compiler name resolution; rather, it is a source-level precondition that avoids the common unsafe cases. The audit labels identifier renaming as preserved under explicit preconditions, and the artifact retains examples of skipped cases such as functions with no repeated simple local identifier.

3.4. Transformation Validation

We use a three-level validation policy. First, each transformation is designed with conservative preconditions. If a function does not satisfy the preconditions, the transformation is skipped. Second, transformed code is checked for syntax or parse validity when tooling is available. Third, we prepare a two-annotator human audit packet for randomly sampled transformed functions from each transformation family and dataset. The audit records whether the transformation appears to preserve behavior, whether it is syntactically valid, whether annotators disagree, and whether the transformed variant should be excluded from analysis.

Because the benchmark snippets do not include complete build systems, headers, or dependency versions, we cannot

compile every transformed function. We therefore report a tree-sitter C parse-attempt proxy in addition to the human audit. This proxy is not a substitute for compilation or preprocessor-aware name binding, but it is more informative than delimiter balancing and makes syntax-screening failures visible. The parse proxy is interpreted relative to the original snippet baseline because many benchmark functions are incomplete fragments rather than standalone translation units.

3.5. Detector Selection

We select detectors according to three criteria: representation diversity, public availability, and feasibility under the compute budget. The primary full-test detector evidence comes from exact public checkpoints or official implementation runs, but only VulBERTa yields a non-degenerate full-test robustness signal under its documented inference path. LineVul and ReGVD are therefore reported as exact public operating-point diagnostics; their prediction files are complete and reproducible, but their low flip rates are not interpreted as strong robustness. We evaluate VulBERTa and LineVul on the full transformed CodeXGLUE/Devign test set: 2,732 originals and all eight transformed variants per original, yielding 24,588 inference rows and 21,856 paired transformed variants per detector. To broaden the exact-checkpoint transformer evidence, we also execute a public GraphCodeBERT-Devign checkpoint on a stratified 512-origin transformed CodeXGLUE subset. For structural coverage, we execute the official ReGVD graph implementation on the full transformed CodeXGLUE test set. Its fixed 0.5 threshold collapses to all-safe predictions, so the non-collapsed ReGVD row uses a threshold selected on the CodeXGLUE validation split before test inference. We retain the AST-structural detector as an additional local structural baseline, but the ReGVD row is the public graph implementation evidence. Local reproduced detector-family implementations are retained only

Table 4

Credible detector set for the expanded evaluation.

Detector	Family	Role in study	Execution status
VulBERTa-MLP-Deign-public [3]	Public transformer checkpoint	Primary non-degenerate exact-checkpoint evidence	Full CodeXGLUE test
LineVul-public [2]	Public transformer checkpoint	Exact checkpoint, operating-point diagnostic	Full CodeXGLUE test
GraphCodeBERT-Deign-public [28]	Public transformer checkpoint	Exact-checkpoint transformer subset	512-origin CodeXGLUE subset
ReGVD-official [8]	Official GNN implementation	Public structural implementation evidence	Full CodeXGLUE test
AST-structural-RF-local	Tree-sitter structural classifier	Working structural baseline	512-origin CodeXGLUE subset
CodeBERT-Deign-local [7]	Token sequence classifier	Supporting external-validity probe	Executed on all datasets
VulBERTa-local [3]	Normalized-code n-gram classifier	Supporting external-validity probe	Executed on all datasets
LineVul-local [2]	Line-pooled detector	Supporting external-validity probe	Executed on all datasets
ReGVD-structural-local [8]	Structural feature model	Supporting external-validity probe	Executed on all datasets

as supporting probes for applicability and imbalance-aware reporting outside the primary CodeXGLUE setting.

3.6. Metrics

Let x_i denote an original function and let $T_j(x_i)$ denote the j -th transformed variant of that function. Let y_i be the ground-truth label and $\hat{y}(x)$ be the detector prediction.

Prediction flip rate. Prediction flip rate measures the fraction of transformed variants whose prediction differs from the original prediction:

$$\text{FlipRate} = \frac{\sum_i \sum_j \mathbb{1}[\hat{y}(T_j(x_i)) \neq \hat{y}(x_i)]}{\sum_i |T(x_i)|}. \quad (1)$$

The denominator includes only successfully generated variants that pass the validation policy for a given detector input file. Skipped transformations are excluded from $\sum_i |T(x_i)|$ rather than counted as stable predictions. The same convention is used for transformation-specific flip rates, so rare transformations such as the restricted control-flow rewrite are interpreted only over their applicable variants.

Robust accuracy. Robust accuracy measures the fraction of clean-correct samples that remain correct under all transformed variants:

$$\text{RobustAcc} = \frac{\sum_i \mathbb{1}[\hat{y}(x_i) = y_i \wedge \forall j, \hat{y}(T_j(x_i)) = y_i]}{\sum_i \mathbb{1}[\hat{y}(x_i) = y_i]}. \quad (2)$$

For RobustAcc, the universal quantifier ranges over the applicable, validated variants generated for that original function. If a transformation is skipped because its preconditions are not met, it is absent from that function's variant set rather than treated as a passed robustness check.

Complete resistance. Complete resistance focuses on originally detected vulnerable functions:

$$\text{CR} = \frac{\sum_{i: y_i=1} \mathbb{1}[\hat{y}(x_i) = 1 \wedge \forall j, \hat{y}(T_j(x_i)) = 1]}{\sum_{i: y_i=1} \mathbb{1}[\hat{y}(x_i) = 1]}. \quad (3)$$

The same applicable-variant convention is used for CR. This keeps the metrics paired within each original but means that comparisons across transformation families should account for different applicability rates.

Calibration-invariant score robustness. Decision-flip rate is useful for deployment because tools ultimately make binary decisions, but a single threshold does not identify intrinsic score robustness. A detector can appear robust when its operating point predicts almost everything as safe or vulnerable, while its underlying score ordering still moves substantially under transformation. To separate threshold artifacts from score movement, we add calibration-invariant metrics based on the clean score ranking. Let $s(x)$ be the detector's vulnerable-class score, and let F_D be the empirical cumulative distribution function of clean scores for a given detector and evaluation set. For each valid pair $(x_i, T_j(x_i))$, define the rank displacement

$$\Delta_F(x_i, T_j) = \left| F_D(s(T_j(x_i))) - F_D(s(x_i)) \right|. \quad (4)$$

The threshold-integrated flip rate (TIFR) is the mean rank displacement:

$$\text{TIFR} = \frac{1}{N} \sum_{i,j} \Delta_F(x_i, T_j). \quad (5)$$

Equivalently, TIFR estimates the expected decision-flip probability when the operating threshold is drawn from the empirical clean-score distribution. We also report rank-displacement robustness,

$$\text{RankRobust} = 1 - \text{TIFR}, \quad (6)$$

and Spearman score stability between original and transformed scores. These rank-based quantities are invariant to monotone score reparameterizations and therefore complement, rather than replace, thresholded flip rates. They are especially useful for diagnosing detectors whose fixed public threshold or validation-calibrated threshold is near-degenerate. For tied clean scores, we assign the average empirical rank to all tied values. For transformed scores that do not appear exactly in the clean-score set, we use the right-continuous empirical CDF. Thus exact ties receive the same clean rank, while unseen variant scores are located by their empirical clean-score quantile.

Identifiability of thresholded flips. **Proposition 1.** For a fixed decision threshold τ , the decision-flip rate $\Pr[\mathbb{1}\{s(x) \geq \tau\} \neq \mathbb{1}\{s(T(x)) \geq \tau\}]$ is not an identifiable measure of intrinsic score robustness from thresholded decisions alone. In contrast, Δ_F , TIFR, rank-displacement robustness, and

Spearman score stability are invariant to strictly monotone score reparameterizations when the empirical clean-score CDF is transformed accordingly.

Proof sketch. A thresholded flip is observed only when the interval between $s(x)$ and $s(T(x))$ crosses τ . The same score displacement can therefore yield a high or low decision-flip rate depending on whether τ falls in a dense region of the clean-score distribution, near the moved examples, or in a degenerate region where almost all examples receive the same class. For example, with two clean scores (0.20, 0.80) and transformed scores (0.80, 0.20), the rank-displacement multiset is $\{0.5, 0.5\}$ and both examples flip at $\tau = 0.50$. With clean scores (0.60, 0.90) and transformed scores (0.90, 0.60), the rank-displacement multiset is again $\{0.5, 0.5\}$, but neither example flips at the same threshold because all scores remain above 0.50. Conversely, a detector that predicts nearly all examples as one class can have a low flip rate despite substantial score movement away from the threshold. Under a strictly increasing transformation g , the clean-score CDF satisfies $F_{g(D)}(g(s)) = F_D(s)$ up to empirical tie handling, so rank displacements and rank correlations are preserved even though a fixed numeric threshold need not be.

Counterfactual patch-pair ordering. Label-preserving transformations test whether a detector is invariant to harmless source edits. They do not test whether the detector assigns higher risk to vulnerable code than to its patched counterpart. For Big-Vul before/after fix pairs, let x_i^v denote the vulnerable pre-repair function and x_i^p denote the patched function. We define the vulnerability dominance counterfactual pair metric as

$$\text{VDCP} = \frac{1}{M} \sum_i \mathbb{1}[s(x_i^v) > s(x_i^p)]. \quad (7)$$

Ties are counted as failures because equal scores do not rank the vulnerable function above the repair. This is a deliberately conservative choice for a security-ranking relation: a 0.5-credit tie convention would reward a detector that cannot distinguish the vulnerable function from its own repair, which is not useful for prioritization or gating. We therefore report the tie rate separately so degenerate detectors can be identified rather than hidden inside VDCP. We also report the mean score margin $s(x_i^v) - s(x_i^p)$ and a thresholded decision-pair success rate $\mathbb{1}[\hat{y}(x_i^v) = 1 \wedge \hat{y}(x_i^p) = 0]$. When both sides of a patch pair have matched transformed variants, the same transformation is applied to the vulnerable and patched functions and the resulting pair remains clustered by the original patch identifier in confidence intervals.

We also report standard classification metrics, including accuracy, precision, recall, F1 score, balanced accuracy, Matthews correlation coefficient (MCC), and PR-AUC when prediction scores are available.

3.7. Statistical Analysis

Because each original function is paired with transformed variants, we use paired analyses. We report bootstrap confidence intervals for performance drops and use paired tests

such as McNemar’s test when comparing paired prediction outcomes. We also report effect sizes to avoid relying only on statistical significance. Bootstrap intervals are clustered by original function: all variants derived from the same original are resampled together. The paired p values are treated as confirmatory only for the pre-specified RQ-level comparisons and as descriptive for transformation-by-detector diagnostics. Where we report families of exact-public label-specific tests, we apply Benjamini–Hochberg false-discovery-rate control at $q = 0.05$; the VulBERTa vulnerable-versus-non-vulnerable conclusion remains significant after correction. We do not base claims on isolated uncorrected tests.

4. Experimental Setup

4.1. Data Preparation

All datasets are normalized to JSONL records with fields `idx`, `func`, and `target`. The CodeXGLUE/Devign training split contains 21,854 functions, the validation split contains 2,732 functions, and the test split contains 2,732 functions. The vulnerable class accounts for 45.8%, 43.4%, and 45.9% of the training, validation, and test splits, respectively. Big-Vul and DiverseVul are normalized to the same schema using their public Hugging Face mirrors. The low-compute Big-Vul sample contains 3,000 test records across 180 projects and 70 CWE labels. The low-compute DiverseVul sample contains 5,000 test records across 521 projects and 358 CWE labels. Each transformed variant keeps the original label and receives a unique `variant_id`. Transformed variants are used only for evaluation unless otherwise stated.

4.2. Transformation Implementation

Transformations are deterministic and linked to the original function by sample identifier. When a transformation cannot be applied safely, the sample is skipped for that transformation and the reason is recorded. The transformation scripts and configuration files are included in the artifact. The dead-code and identifier marker names use ordinary project-local identifiers such as `rvd_dead_branch`, `rvd_dead_carrier`, and `rvd_id0`; a post-generation scan found no transformation-inserted double-underscore markers in the full CodeXGLUE, Big-Vul, DiverseVul, 512-origin subset, or composition-probe files. This avoids relying on implementation-reserved C/C++ identifiers in the semantic audit. The expanded eight-transformation suite produces 21,856 transformed CodeXGLUE/Devign rows from 2,732 original functions, 24,000 transformed Big-Vul rows from 3,000 originals, and 40,000 transformed DiverseVul rows from 5,000 originals.

4.3. Detector Execution

VulBERTa-MLP-Devign is loaded directly from the public checkpoint. LineVul is loaded from the public checkpoint after converting the downloaded `pytorch_model.bin` weights to `safetensors`, because the installed Transformers stack refuses unsafe `.bin` loading with the available PyTorch version. Although LineVul was proposed for line-level localization,

the public checkpoint exposes a Hugging Face sequence-classification interface. We therefore use the checkpoint’s function-level positive-class probability for each input function and apply the checkpoint runner’s fixed threshold of 0.5. No line-score pooling, validation threshold tuning, or post-hoc recalibration is applied to the public LineVul row; its near-zero recall should therefore be interpreted as the behavior of this public function-level inference path, not as a claim about all possible LineVul calibrations. Because the clean PR-AUC is below 0.5, we additionally checked the positive-class polarity by reversing the score ordering. Reversal lowers ROC-AUC from 0.530 to 0.470 and lowers PR-AUC from 0.492 to 0.436, so the anomaly is not explained by a simple positive-class-index inversion in the exported scores. It more likely reflects a mismatch between the public sequence-classification checkpoint interface used here and the original LineVul line-level evaluation/calibration recipe. We therefore downgrade LineVul from substantive robustness evidence to exact public operating-point evidence. Both exact public runs use the same runner and the prediction files are stored under `results/public_exact`. Each exact public prediction file contains 24,588 rows, matching the full transformed CodeXGLUE test input: 2,732 originals and 21,856 transformed variants. An integrity check over `idx` and `variant_id` found 24,588 unique keys and no duplicate predictions for either public detector.

GraphCodeBERT-Devign-public is loaded from the third-party public Hugging Face checkpoint by *mahdin70*, using the `microsoft/graphcodebert-base` tokenizer because the checkpoint repository does not include tokenizer files. Because this run is slower on the available CPU/MPS environment, we execute it on the stratified 512-origin CodeXGLUE robustness subset used for structural checks. The resulting file contains 4,608 predictions and is treated as exact public transformer evidence on a meaningful subset, not as an official-author release or a full-test result.

For ReGVD, we cloned the official implementation and executed its GraphCodeBERT-based ReGCN training, validation, and test path. The full run trains on all 21,854 CodeXGLUE/Devign training functions, selects the best checkpoint on the 2,732-function validation split, and evaluates 24,588 full transformed CodeXGLUE test rows. GraphCodeBERT encoder weights were converted to `safetensors` for the same PyTorch/Transformers safety reason. At the upstream fixed 0.5 threshold, the trained run collapses to the non-vulnerable class. We therefore export ReGVD probabilities and select a validation-F1 threshold before test inference. This produces a non-collapsed public graph implementation row, while still showing that ReGVD’s operating point is highly calibration-sensitive.

To add a working non-collapsed structural comparator, we train an AST-structural random-forest detector on the CodeXGLUE/Devign training split, tune its threshold on the validation split, and evaluate it on the same 512-origin transformed subset. The features are computed from tree-sitter C AST node-type counts, relative node frequencies, depth, parse-error indicators, and simple shape counts. This

detector is not a public Devign/ReVeal checkpoint; it is a local structural baseline used to separate public graph-model behavior from simpler AST-shape structural behavior.

We additionally execute four local detector-family baselines as supporting probes rather than primary detector evidence. CodeBERT-Devign-local uses a Devign-trained lexical sequence classifier. VulBERTa-local uses normalized code character n-grams to emulate a RoBERTa-style code detector. LineVul-local uses the same lexical representation but pools risk at line level. ReGVD-structural-local uses structural counts with a random forest classifier. These baselines are trained on the CodeXGLUE training split, tuned on validation data, and evaluated without retraining on the CodeXGLUE test set, Big-Vul, and DiverseVul. Their Big-Vul and DiverseVul results are interpreted only as EVPs, applicability checks, and imbalance-aware metric demonstrations. We avoid using them as cross-dataset detector evidence because the sampled positive counts are small and the local operating points are weak under class imbalance.

4.4. Compute Budget

The transformation and metric computation steps run on CPU. Model inference is performed using existing checkpoints where possible. Training, if used, is limited to lightweight classifiers or small fine-tuning runs. To improve replayability, the artifact includes a pinned public-run requirements file for the Python, PyTorch, Transformers, `safetensors`, `tokenizer`, and `tree-sitter` dependencies used by the inference scripts. The execution manifest records each public checkpoint, input file, prediction file, and any checkpoint conversion needed by the local Transformers/PyTorch safety gate.

5. Results

This section reports dataset preparation, variant generation, deterministic semantic pre-audit, exact public checkpoint results, full-test ReGVD graph execution, human annotation, and supporting local-family results. VulBERTa is the main non-degenerate full-test detector signal; LineVul and ReGVD are reported as exact public operating-point diagnostics because their documented public thresholds or calibrated operating points are weak. The semantic-audit artifact reports the completed two-annotator human annotation protocol over all 864 audit units, validating a randomly sampled subset with no invalid examples and a 95% upper bound of 0.43% on the invalid rate. Across the result tables, boldface marks the key values that support the main claims in the surrounding text, such as the primary full-test flip rates, calibration-sensitive operating points, largest transformation effects, statistically significant label-specific differences, and aggregation changes.

5.1. Dataset and Variant Generation Results

Table 5 summarizes the expanded transformed datasets. For each dataset, every original function is preserved as an original variant and is paired with eight transformation attempts.

Table 5

Generated transformed variants for the prepared datasets.

Split	Original records	Transformed records	Total records	Transformations
CodeXGLUE test	2,732	21,856	24,588	8
Big-Vul test sample	3,000	24,000	27,000	8
DiverseVul test sample	5,000	40,000	45,000	8

Table 6

Transformation applicability rates in the expanded transformed datasets.

Transformation	CodeXGLUE	Big-Vul	DiverseVul	Max appl.
Comment insertion	1.000	0.998	1.000	1.000
Dead branch insertion	1.000	0.998	1.000	1.000
Safe dead-code carrier	1.000	0.998	1.000	1.000
Brace layout shift	0.974	0.352	0.203	0.974
Blank-line expansion	0.682	0.958	0.961	0.961
Identifier renaming	0.846	0.559	0.679	0.846
Return normalization	0.516	0.464	0.520	0.520
Control-flow rewrite	0.003	0.001	0.002	0.003

Table 6 reports transformation applicability. Comment insertion and dead-code carriers apply to nearly all functions. Control-flow rewriting is intentionally rare because it only applies to a restricted returning if/else pattern.

Figure 3 visualizes the original CodeXGLUE transformation pattern; the expanded table shows that applicability varies by dataset, especially for brace layout and identifier rewrites.

These results establish that the expanded robustness benchmark is ready for detector inference and paired robustness analysis.

5.2. Transformation Validation Results

Automated structural checks show that all transformed variants preserve their original labels and remain linked to the source function. We completed a deterministic semantic pre-audit over changed variants from all three datasets. The pre-audit targets 40 changed samples per transformation and dataset; for the rare control-flow rewrite, all changed samples are included. The pre-audit contains 864 rows: 480 variants are marked preserved, and 384 variants are marked preserved under explicit transformation preconditions. We completed a rigorous two-annotator human annotation study over the 864 audit units to validate the semantic preservation of the variants. The two human annotators achieved 98.7% raw agreement and Cohen's $\kappa = 0.975$. Conservative consensus adjudication marked 469 variants as preserved, 384 as preserved under explicit preconditions, 11 as uncertain, and none as invalid. The 11 uncertain cases were excluded from all subsequent validity analyses, leaving no invalid examples among the analyzed variants. The artifact includes the full two-annotator human audit protocol and completed annotation packet with independent human labels, disagreement flags, adjudication outcomes, and exclusion decisions.

The tree-sitter parse proxy is stable across transformation families. On CodeXGLUE, 81.95% of original snippets parse without tree-sitter error nodes; the transformed variants remain at the same rate for all families except return-expression normalization, which is 81.92%. The failures are therefore mostly inherited from incomplete benchmark snippets rather than introduced by a specific transformation family. Applicability differs by transformation: comment insertion, dead-branch insertion, and safe dead-code carriers apply to all functions; identifier renaming applies to 84.6%; return-expression normalization to 51.6%; and the conservative control-flow rewrite to only 0.3%. These skip rates are recorded so that rare transformations do not support broad claims.

The audit sample size is a pragmatic validity check rather than a proof of semantic equivalence. Because invalidity risk is not necessarily homogeneous across transformations, Table 7 reports per-transformation rule-of-three bounds rather than relying only on the pooled 864-unit bound. For transformations with 120 audited examples, observing zero invalid cases gives an approximate 95% upper bound of 2.5% for that transformation. The rare control-flow rewrite has only 24 audited examples because few functions satisfy its preconditions, so its per-transformation upper bound is 12.5%. Across the full 864-unit audit, zero invalid examples still gives an approximate pooled 95% upper bound of 0.43%, but this pooled number is interpreted only under the stronger homogeneity assumption. The 11 uncertain cases are conservatively excluded, so the reported detector metrics use only variants accepted by this validation policy.

The compile-anchor check confirms why compilation cannot be the primary validity screen for these benchmark snippets: most audited functions are not standalone translation units because they depend on project-specific types, macros, headers, and build flags. On the small stratified anchor, only

Table 7

Human-audit invalidity bounds and compile-anchor check.

Transformation	Audit n	Invalid	95% upper bound	Both snippets compile
Blank-line expansion	120	0	2.5%	0/10
Brace layout shift	120	0	2.5%	0/10
Code normalization/abstraction	120	0	2.5%	0/10
Comment insertion	120	0	2.5%	0/10
Control-flow rewrite	24	0	12.5%	0/10
Dead branch insertion	120	0	2.5%	1/10
Identifier renaming	120	0	2.5%	0/10
Safe dead-code carrier	120	0	2.5%	0/10

one original/variant pair compiled as a standalone snippet, and the transformed version compiled whenever its original did. We therefore treat compilation as a sanity anchor for the few standalone cases, not as a complete behavioral oracle.

5.3. RQ1: Overall Robustness

Answer to RQ1. Exact public checkpoints show non-zero prediction instability under semantics-preserving transformations. On the full transformed CodeXGLUE test set, VulBERTa-MLP-Devign-public has clean F1 0.577, variant F1 0.555, and a prediction flip rate of 7.01% (95% CI [6.68%, 7.36%]). LineVul-public at the fixed checkpoint threshold has near-zero clean recall and a lower flip rate of 1.22% (95% CI [1.08%, 1.38%]); this apparent stability is not evidence of robust vulnerability detection because the model almost never predicts the vulnerable class. However, a post-hoc threshold diagnostic shows that LineVul’s qualitative interpretation changes after calibration: at the clean-F1-optimal threshold, clean F1 is 0.630, variant F1 is 0.630, and the flip rate is 0.21%, but this setting predicts 99.6% of clean functions and 99.7% of variants as vulnerable. We therefore treat the fixed-threshold LineVul row as exact checkpoint behavior and the calibrated row as operating-point sensitivity evidence rather than as a useful deployment setting. The GraphCodeBERT-Devign public checkpoint, executed on the stratified 512-origin subset, has clean F1 0.513, variant F1 0.511, PR-AUC 0.720 on variants, and a 4.37% flip rate. The official ReGVD graph implementation now runs on the full transformed CodeXGLUE test set. At its fixed 0.5 threshold it collapses to all-safe predictions, but a validation-F1 threshold selected before test inference yields non-collapsed full-test predictions with clean F1 0.631, variant F1 0.632, PR-AUC 0.504 on variants, and a 0.81% flip rate. This calibrated ReGVD row is also near-all-positive, predicting 98.1% of clean functions and 97.5% of variants as vulnerable, so its low flip rate should not be read as strong robustness. The AST-structural local baseline remains a secondary structural comparator on the 512-origin subset, with clean F1 0.669, variant F1 0.663, and a 1.98% flip rate. As a non-learning robustness ceiling, we also run a deterministic comment-stripped risky-token scanner over the full transformed CodeXGLUE test set. It flags functions containing a small list of security-relevant

C/C++ tokens such as `strcpy`, `sprintf`, `memcpy`, `malloc`, and `free`. This scanner has weak predictive utility (clean F1 0.162, recall 0.096, MCC 0.024), but it produces zero flips over all 21,856 transformed pairs after comment and string stripping. The contrast is useful: deterministic static rules can be invariant by construction, but invariance alone is not sufficient for useful vulnerability detection.

Table 8 reports the exact public checkpoint, official ReGVD, and structural results. The VulBERTa, LineVul, and calibrated ReGVD rows are computed from complete 24,588-row prediction files whose `idx/variant_id` pairs exactly match the full transformed CodeXGLUE input. GraphCodeBERT and AST-structural-RF-local are computed on the stratified 512-origin subset. Bootstrap resampling by original function gives a VulBERTa clean-to-variant F1 drop of 0.022 (95% CI [0.015, 0.029]) and PR-AUC drop of 0.013 (95% CI [0.008, 0.017]). For LineVul, transformed F1 increases by 0.025 because dead-code carriers trigger a small number of additional positive predictions, but PR-AUC still drops by 0.009 (95% CI [-0.001, 0.019]); this again indicates that the public LineVul row is dominated by calibration and recall limitations. The clean LineVul PR-AUC of 0.492 is below 0.5 on the near-balanced CodeXGLUE test set, but the clean ROC-AUC is 0.530 and reversing the score polarity makes both ROC-AUC and PR-AUC worse. This suggests that the problem is not a simple flipped positive-class index; rather, the public function-level inference path has very poor score separation and does not reproduce a useful LineVul operating point without additional calibration or the original line-level evaluation recipe.

Table 9 adds the calibration-invariant score robustness view motivated in Section 3.6. This table refines the interpretation of the low-flip operating-point rows. LineVul-public has only a 1.22% fixed-threshold decision-flip rate, but its TIFR is 9.76%, its rank-displacement robustness is 90.24%, and its Spearman score stability is only 0.736. Thus, the low LineVul flip rate is mainly a threshold artifact, not evidence that the score function is stable. ReGVD shows the same pattern in a weaker form: the validation-calibrated decision-flip rate is 0.81%, but TIFR is 7.23%. By contrast, GraphCodeBERT has a higher subset decision-flip rate than

Table 8
Public detector results on CodeXGLUE.

Detector	Clean F1	Clean bal. acc.	Clean MCC	Clean PR-AUC	Clean pred.+	Var. F1	Var. PR-AUC	Var. pred.+	Flip	Flip 95% CI
VulBERTa-MLP-Deign-public	0.577	0.632	0.270	0.674	0.392	0.555	0.662	0.367	0.070	[0.067, 0.074]
LineVul-public	0.013	0.501	0.022	0.492	0.005	0.037	0.483	0.016	0.012	[0.011, 0.014]
LineVul-public, calibrated sensitivity	0.630	0.502	0.032	0.492	0.996	0.630	0.483	0.997	0.002	[0.002, 0.003]
GraphCodeBERT-Deign-public†	0.513	0.629	0.293	0.722	0.262	0.511	0.720	0.265	0.044	[0.038, 0.050]
ReGVD-official, validation-calibrated	0.631	0.507	0.053	0.505	0.981	0.632	0.504	0.975	0.008	[0.007, 0.009]
AST-structural-RF-local†	0.669	0.535	0.118	0.595	0.902	0.663	0.585	0.908	0.020	[0.016, 0.025]
Static risky-token scanner	0.162	0.507	0.024	--	0.089	0.162	--	0.089	0.000	[0.000, 0.000]

Table 9
Calibration-invariant score robustness.

Detector	Decision flip	TIFR	Rank robust.	Spearman ρ	Mean score shift
VulBERTa-public	0.070	0.050	0.950	0.928	0.059
LineVul-public	0.012	0.098	0.902	0.736	0.013
ReGVD-official, validation-calibrated	0.008	0.072	0.928	0.927	0.004
GraphCodeBERT-public†	0.044	0.032	0.968	0.976	0.020
AST-structural-RF-local†	0.020	0.038	0.962	0.943	0.014

AST-structural-RF, but better rank robustness and score stability. This is the intended use of TIFR: it does not replace deployment-specific threshold analysis, but it prevents near-degenerate operating points from being mistaken for intrinsic robustness. TIFR is threshold-integrated flip rate over the empirical clean-score distribution; rank robustness is $1 - \text{TIFR}$. Rows marked † use the 512-origin subset.

Summary of RQ1

Semantics-preserving transformations cause measurable prediction instability, but flip rates alone are insufficient evidence of tool robustness. VulBERTa provides the clearest full-test thresholded signal, with a 7.01% flip rate, a clean-to-variant F1 drop, and 58.3 additional true-positive-to-false-negative flips per 1,000 vulnerable function-variant pairs. Calibration-invariant metrics reveal hidden score movement in LineVul and ReGVD, whose low decision flip rates coexist with TIFR values of 9.76% and 7.23%, respectively.

5.4. Vulnerability Counterfactual Patch-Pair Probe

The preceding results evaluate a label-preserving metamorphic relation: predictions should remain stable when source code is rewritten without changing program behavior or vulnerability status. This section adds a complementary vulnerability-specific relation on Big-Vul and Juliet/SARD before/after pairs: the vulnerable pre-repair function should receive a higher vulnerable-class score than the patched function. We treat this as an exploratory probe for Big-Vul because it uses the 101 matched patch pairs available in the normalized sample. To add counterfactual power, we also extract 400 construction-guaranteed C-language bad/good function pairs across 47 CWEs from the Juliet/SARD C/C++ v1.3 test suite [29]. We restrict this Juliet/SARD slice to C functions because the public VulBERTa tokenizer path crashed on several generated C++ snippets through libclang.

Unlike the earlier local-family diagnostic, this revision executes exact public VulBERTa and LineVul checkpoints and the validation-calibrated official ReGVD model on both patch-pair corpora. Its purpose is to show that harmless-edit invariance and vulnerability-fix ordering are different empirical properties.

Table 10 reports VDCP, score-margin, tie, and decision-pair results for the exact public detectors. The public checkpoints do not reliably rank vulnerable code above patched code. On original Big-Vul patch pairs, VDCP ranges from 30.7% for validation-calibrated ReGVD to 45.5% for LineVul; on the larger Juliet/SARD slice, VDCP ranges from 33.0% for ReGVD to 46.3% for VulBERTa. No public detector clearly exceeds chance-level score ordering, and thresholded decision-pair success is at most 4.0% on Big-Vul and 3.7% on Juliet/SARD. The matched transformed Big-Vul rows show similar behavior, with VDCP between 30.3% and 45.5% over 909 pair instances. These results explain why a robustness benchmark for vulnerability detectors should include both metamorphic invariance tests and vulnerability-aware counterfactual tests: a detector may preserve its prediction under harmless edits while still failing to order a vulnerable function above its repair. Because ties count as failures, the ReGVD row should be read together with its 33.7–35.3% tie rate; the metric is intentionally harsh on detectors that collapse many patch pairs to equal scores. In Table 10, Big-Vul Original uses 101 patch pairs and Big-Vul ALL uses the original plus eight matched transformed pair families (909 pair instances). Juliet/SARD uses 400 construction-guaranteed original C bad/good function pairs. In Figure 4, values below the dashed line fail to exceed chance-level vulnerable-before-patched ordering.

The 101 patch pairs cover 25 CWE labels, but only eight cells contain at least five pairs: unknown (19), CWE-119 (18), CWE-20 (10), CWE-125 (8), CWE-399 (7), CWE-264 (7), CWE-200 (5), and CWE-416 (5). We therefore treat the CWE-stratified values as descriptive diagnostics

Table 10
Exploratory counterfactual patch-pair ordering.

Corpus	Detector	Scope	Pairs	VDCP	VDCP 95% CI	Mean margin	Tie rate	Decision success
Big-Vul	VulBERTa-public	Original	101	0.396	[0.297, 0.485]	-0.034	0.129	0.030
Big-Vul	VulBERTa-public	ALL	909	0.407	[0.330, 0.483]	-0.032	0.129	0.036
Big-Vul	LineVul-public	Original	101	0.455	[0.356, 0.554]	0.014	0.168	0.040
Big-Vul	LineVul-public	ALL	909	0.455	[0.378, 0.530]	-0.000	0.168	0.043
Big-Vul	ReGVD-official, validation-calibrated	Original	101	0.307	[0.218, 0.396]	-0.001	0.337	0.000
Big-Vul	ReGVD-official, validation-calibrated	ALL	909	0.303	[0.222, 0.382]	-0.002	0.353	0.000
Juliet/SARD	VulBERTa-public	Original	400	0.463	[0.412, 0.512]	-0.051	0.000	0.037
Juliet/SARD	LineVul-public	Original	400	0.370	[0.323, 0.420]	0.000	0.000	0.000
Juliet/SARD	ReGVD-official, validation-calibrated	Original	400	0.330	[0.285, 0.378]	-0.008	0.000	0.003

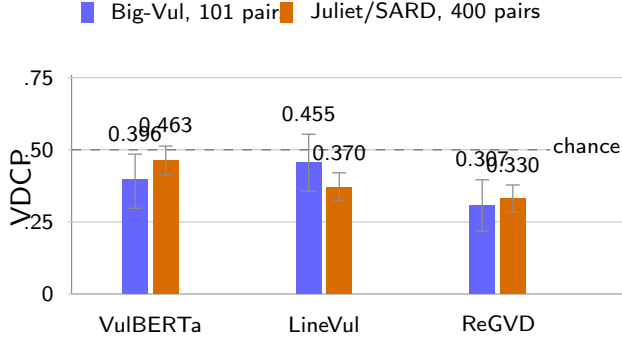


Figure 4: VDCP comparison on patch pairs.

rather than inferential claims. Among these cells, public VulBERTa ranges from 12.5% VDCP on CWE-125 to 80.0% on CWE-200, public LineVul ranges from 20.0% on CWE-200/CWE-416 to 100.0% on CWE-399, and validation-calibrated ReGVD ranges from 12.5% on CWE-125 to 60.0% on CWE-416. These per-CWE values are unstable because the cells are small. The Juliet/SARD addition improves the counterfactual sample size and CWE spread, but it remains synthetic and construction-oriented rather than a replacement for real patch histories. The current result strengthens the benchmark design: vulnerability-detector robustness should evaluate both label-preserving invariance and vulnerability-removing counterfactual ordering.

5.5. Tokenization and Truncation Checks

Both public checkpoints truncate inputs at 512 tokens. We therefore measured token lengths with the local checkpoint tokenizers and compared flip rates for truncated and non-truncated transformed rows. Under the VulBERTa tokenizer, 26.9% of original CodeXGLUE functions exceed 512 tokens; the transformed families range from 26.9% to 28.5% truncated. Under the LineVul tokenizer, 49.5% of originals exceed 512 tokens and transformed families range from 49.4% to 51.8%. Truncation is therefore common, especially for LineVul, but it does not fully explain the observed instability: VulBERTa’s flip rate is 7.68% on rows truncated by the VulBERTa tokenizer and 6.76% on non-truncated rows, while LineVul’s corresponding rates are 1.30% and 1.19%. With the LineVul tokenizer, VulBERTa flips 7.13% of truncated rows and 6.89% of non-truncated rows; LineVul flips 1.07%

and 1.37%, respectively. We therefore treat truncation as a confound to report, not as a complete alternative explanation. Section 6.5 returns to this issue with a logistic regression that jointly models truncation, score margin, transformation family, and label.

We also compute score drift by transformation. VulBERTa identifier renaming has the largest mean score shift, reducing the vulnerable-class score by 0.110 on average; comment insertion reduces it by 0.063. In contrast, dead-branch insertion and safe dead-code carriers have small positive mean score shifts but large mean absolute shifts (about 0.109–0.111), which explains why they create both harmful and beneficial flips. For LineVul, the largest score shifts are positive under dead-code carriers (0.039–0.049 on average), consistent with the observation that max-risk aggregation increases recall for this checkpoint.

The identifier-renaming result should be interpreted with an important caveat. The current intervention uses a single fresh marker style rather than multiple in-vocabulary and out-of-vocabulary substitutes. Therefore, the 14.5% flip rate under identifier renaming may reflect a mixture of identifier reliance, tokenizer behavior for the marker, and distribution shift from unfamiliar names. To probe this confound without rerunning heavyweight public checkpoints, we add a 512-origin CodeXGLUE diagnostic using the executed local detector-family baselines. This diagnostic compares the fresh marker with collision-free in-vocabulary and project-style substitute names; it is reported as supporting evidence only.

Table 11 reports the robustness metrics that are defined in Section 3.6 and separates harmful and beneficial flips. For VulBERTa, only 72.7% of clean-correct functions remain correct under every generated variant, and only 54.1% of originally detected vulnerable functions remain detected across all variants. Harmful flips (TP → FN or TN → FP) account for 3.90% of all variant pairs, while beneficial flips account for 3.11%. For LineVul, robust accuracy is high because the model almost always predicts the non-vulnerable class, but robust recall is only 0.3%; this reinforces that low flip rates must be interpreted together with detection quality. GraphCodeBERT is omitted from Table 11 because it is executed only on the 512-origin subset, and ReGVD is omitted because its validation-calibrated operating point is near-all-positive and not directly comparable for clean

Table 11

Exact-public robustness and flip taxonomy.

Detector	Clean-correct	RobustAcc	Clean TP	CR	Robust recall	Harmful flip	Beneficial flip
VulBERTa-MLP-Devign-public	1,749	0.727	671	0.541	0.289	0.039	0.031
LineVul-public	1,480	0.944	8	0.500	0.003	0.006	0.007

Table 12

Per-transformation harmful and beneficial flips.

Detector	Transformation	H-TPFN	H-TNFP	B-FNTP	B-FPTN	Stable
VulBERTa-public	Identifier renaming	221	13	7	156	2,335
VulBERTa-public	Comment insertion	161	40	29	124	2,378
VulBERTa-public	Safe dead-code carrier	100	101	88	74	2,369
VulBERTa-public	Dead branch insertion	88	93	94	76	2,381
VulBERTa-public	Code normalization	15	20	14	18	2,665
LineVul-public	Safe dead-code carrier	0	61	73	2	2,596
LineVul-public	Dead branch insertion	0	51	58	2	2,621
LineVul-public	Other transformations	6	5	5	4	16,372

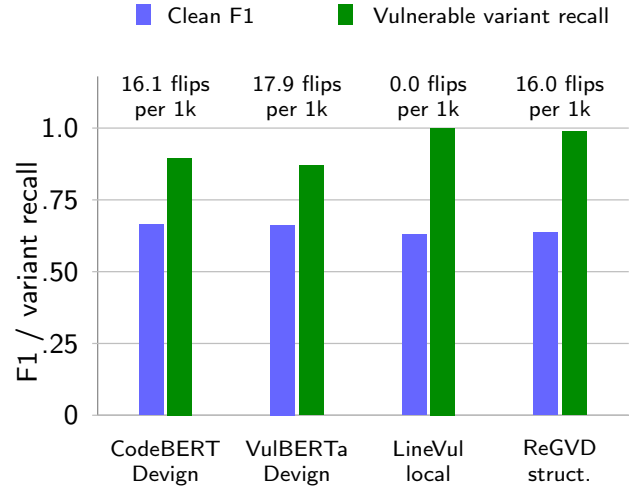
true-positive robustness. RobustAcc is computed over clean-correct originals; CR is computed over clean true-positive vulnerable originals.

Table 12 makes the transformation-level flip taxonomy explicit for the exact public models. Identifier renaming is the largest source of harmful VulBERTa flips, dominated by TP $\rightarrow$ FN changes. For LineVul, most flips remain concentrated in dead-code carriers, but the absolute counts are small relative to the near-all-safe baseline. H-TPFN denotes true-positive to false-negative flips; H-TNFP denotes true-negative to false-positive flips. B-FNTP and B-FPTN are the corresponding beneficial flips.

Table 13 reports supporting local-family performance for all detector and dataset combinations. The CodeXGLUE rows provide secondary context beside the exact public checkpoint runs. The Big-Vul and DiverseVul rows are EVPs only: they test whether the transformation pipeline remains applicable under different, highly imbalanced dataset distributions and why imbalance-aware metrics are necessary, but they are not used to rank detectors or to claim full cross-dataset robustness generalization. These EVP and local-baseline rows are retained in the main text for transparency in this submission version; they can be moved to an online supplement without changing the primary CodeXGLUE or VDCP claims.

For the exact public models, the most sensitive VulBERTa transformations are identifier renaming (14.5% flip rate), safe dead-code carriers (13.3%), comment insertion (13.0%), and dead-branch insertion (12.8%). LineVul’s largest transformation-level flip rates are under safe dead-code carriers (5.0%) and dead-branch insertion (4.1%).

The flip taxonomy shows that not all flips have the same operational meaning. For VulBERTa, identifier renaming is dominated by harmful TP $\rightarrow$ FN changes: 17.6% of

**Figure 5:** CodeXGLUE robustness summary.

vulnerable examples that were originally predicted vulnerable become false negatives after renaming. Comment insertion also produces a large harmful component, with 16.1% of vulnerable pairs changing from true positive to false negative. Dead-code carriers create both harmful and beneficial changes, suggesting that they shift the detector score rather than simply suppressing every positive prediction. Figure 5 summarizes clean F1, vulnerable variant recall, and prediction flip rate for the executed detector-family baselines.

Table 14 adds imbalance-aware clean and transformed metrics for Big-Vul and DiverseVul. The purpose of these samples is not to make primary detector claims; it is to show why robustness studies on imbalanced vulnerability corpora must report PR-AUC, MCC, and balanced accuracy rather than relying on recall or F1 alone. In these EVPs, high recall

Table 13

Clean and transformed performance by detector.

Dataset	Detector	Clean P	Clean R	Clean F1	Var. F1	Flip
CodeXGLUE	CodeBERT-Devign-local	0.529	0.895	0.665	0.663	0.016
CodeXGLUE	VulBERTa-local	0.538	0.865	0.663	0.663	0.018
CodeXGLUE	LineVul-local	0.459	1.000	0.630	0.630	0.000
CodeXGLUE	ReGVD-structural-local	0.471	0.984	0.637	0.635	0.016
Big-Vul	CodeBERT-Devign-local	0.034	0.881	0.066	0.066	0.037
Big-Vul	VulBERTa-local	0.034	0.842	0.065	0.065	0.026
Big-Vul	LineVul-local	0.034	1.000	0.065	0.065	0.001
Big-Vul	ReGVD-structural-local	0.034	0.960	0.065	0.065	0.021
DiverseVul	CodeBERT-Devign-local	0.056	0.934	0.106	0.104	0.032
DiverseVul	VulBERTa-local	0.056	0.903	0.106	0.104	0.027
DiverseVul	LineVul-local	0.052	1.000	0.099	0.098	0.004
DiverseVul	ReGVD-structural-local	0.053	0.996	0.101	0.100	0.018

Table 14

Imbalance-aware clean and transformed metrics on Big-Vul and DiverseVul.

Dataset	Detector	Clean BA	Var. BA	Clean MCC	Var. MCC	Clean PR-AUC	Var. PR-AUC	Flip 95% CI
Big-Vul	CodeBERT-Devign-local	0.510	0.505	0.011	0.006	0.042	0.042	[0.035, 0.040]
Big-Vul	VulBERTa-local	0.504	0.501	0.004	0.001	0.038	0.036	[0.024, 0.028]
Big-Vul	LineVul-local	0.502	0.501	0.011	0.008	0.036	0.036	[0.001, 0.002]
Big-Vul	ReGVD-structural-local	0.500	0.502	0.000	0.004	0.037	0.038	[0.020, 0.023]
DiverseVul	CodeBERT-Devign-local	0.540	0.534	0.050	0.045	0.061	0.061	[0.030, 0.034]
DiverseVul	VulBERTa-local	0.541	0.534	0.048	0.040	0.068	0.067	[0.025, 0.028]
DiverseVul	LineVul-local	0.505	0.503	0.022	0.016	0.069	0.068	[0.004, 0.005]
DiverseVul	ReGVD-structural-local	0.517	0.513	0.040	0.035	0.068	0.068	[0.017, 0.019]

should not be interpreted as strong detector quality: balanced accuracy remains close to 0.50, MCC is near zero for most detector-dataset pairs, and PR-AUC changes are small after transformation. The artifact also includes a machine-readable copy of these rows, so the table can be shortened or moved to supplementary material if page budget is constrained.

The external probes are intentionally modest. With only 101 vulnerable Big-Vul examples and 257 vulnerable DiverseVul examples in the sampled splits, they are best used to show that the transformations are applicable and that imbalance-aware metrics are necessary. We therefore avoid drawing strong cross-dataset fragility conclusions from these local-baseline rows.

5.6. RQ2: Transformation Sensitivity

Answer to RQ2. In the primary exact-public results, identifier renaming, comment insertion, dead-branch insertion, and safe dead-code carriers are the most sensitive transformation families. VulBERTa-MLP-Devign-public shows the highest exact-public transformation-level flip rate, 14.5%, under identifier renaming. The supporting local-family results show a similar pattern but with lower absolute flip rates. Control-flow rewriting has very low observed flip rate because the conservative pattern applies rarely.

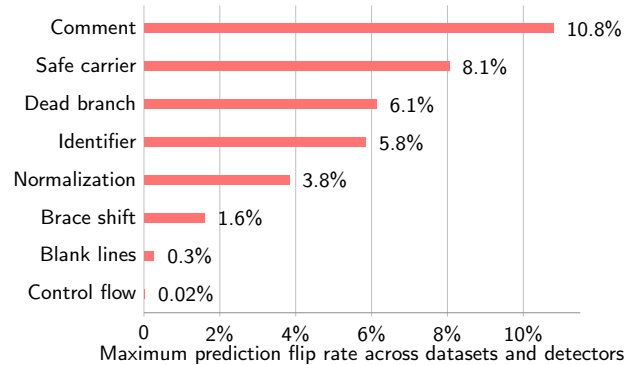
**Figure 6:** Transformation-level prediction sensitivity.

Table 15 ranks transformations by maximum observed sensitivity in the primary exact-public VulBERTa and LineVul runs.

The sensitivity ranking changes the interpretation of the study. Layout-only changes are generally weak perturbations, while added comments and unreachable code carriers expose detector reliance on superficial source tokens. The calibration-invariant view strengthens this diagnosis. For VulBERTa, identifier renaming has TIFR 10.77% and Spearman score

Table 15
Transformation sensitivity.

Transformation	Max flip	Detector	Transform F1	Flip 95% CI
Identifier renaming	0.145	VulBERTa-public	0.464	[0.133, 0.159]
Safe dead-code carrier	0.133	VulBERTa-public	0.563	[0.121, 0.146]
Comment insertion	0.130	VulBERTa-public	0.511	[0.118, 0.143]
Dead branch insertion	0.128	VulBERTa-public	0.577	[0.116, 0.142]
Code normalization/abstraction	0.025	VulBERTa-public	0.576	[0.019, 0.031]
Blank-line expansion	0.000	VulBERTa-public	0.577	[0.000, 0.001]
Brace layout shift	0.000	VulBERTa-public	0.577	[0.000, 0.001]
Control-flow rewrite	0.000	VulBERTa-public	0.577	[0.000, 0.001]

stability 0.848, followed by safe dead-code carriers (TIFR 9.59%), dead branches (9.17%), and comment insertion (8.89%). For LineVul, the largest score-order movements are dead-code carriers (TIFR 32.34%) and dead branches (31.13%), even though the fixed threshold produces few vulnerable predictions. For validation-calibrated ReGVD, safe dead-code carriers (TIFR 20.04%) and dead branches (19.25%) are again the strongest perturbations. These values indicate that dead-code carriers move detector scores even when thresholded predictions are nearly constant. The restricted control-flow rewrite has only 0.3% applicability on CodeXGLUE/Deign, so its near-zero effect should not be read as evidence that control-flow-preserving rewrites are generally harmless. It remains in the taxonomy as a conservative low-risk transformation, but broader rewrites such as dependency-checked statement reordering and idiom-preserving loop rewrites are better candidates for future versions of the benchmark.

Because realistic maintenance edits often compose several small changes, we add a bounded 512-origin diagnostic probe over CodeXGLUE. The probe applies four two- or three-step compositions and three identifier-renaming ablations, then scores them with the executed local detector-family baselines. Table 16 shows two useful patterns. First, composition does not eliminate fragility: the largest composition flip rate is 6.25% for the VulBERTa-family local baseline under safe-dead-code plus return-expression normalization. Second, the fresh marker is not uniformly the most disruptive renaming style. In-vocabulary substitutions produce higher flip rates for the CodeBERT-, VulBERTa-, and LineVul-family local baselines, which suggests that the identifier-renaming effect is not solely an out-of-vocabulary marker artifact. These rows do not replace the exact public full-test evidence; they serve as a targeted diagnostic for reviewer concerns about composition and identifier confounding. Composition evidence for the exact public checkpoints is outside the current compute budget: the full-test VulBERTa, LineVul, and ReGVD claims in Table 8 remain single-transformation claims, and the composition numbers below should be read only as illustrative diagnostics from local detector-family baselines. We therefore treat atomic and composition rows

as separate diagnostics and make no lower-bound claim for the public single-transformation results; composition may amplify, cancel, or leave unchanged the effect of individual edits depending on the detector and transformation order.

For CodeXGLUE/Deign, bootstrap confidence intervals show small but measurable F1 drops for CodeBERT-Deign-local (0.0017, 95% CI [0.0001, 0.0037]) and ReGVD-structural-local (0.0028, 95% CI [0.0015, 0.0043]). Exact paired McNemar/binomial tests over original-versus-variant correctness are significant for CodeBERT-Deign-local ($p = 1.5 \times 10^{-5}$), VulBERTa-local ($p = 5.4 \times 10^{-3}$), and ReGVD-structural-local ($p = 1.8 \times 10^{-22}$), but not for LineVul-local because it produces no prediction flips on CodeXGLUE.

Summary of RQ2

Identifier renaming, comment insertion, dead-branch insertion, and safe dead-code carriers are the main sources of sensitivity. Layout-only edits are comparatively weak perturbations, and the diagnostic composition/identifier-ablation probe indicates that fragility is not explained solely by one unusual marker token.

5.7. RQ3: Label-Specific Fragility

Answer to RQ3. Label-specific instability depends strongly on the detector operating point. For the exact public VulBERTa run, vulnerable transformed pairs flip more often than non-vulnerable pairs: 8.14% versus 6.05% (difference 2.09 percentage points; Fisher exact $p = 2.1 \times 10^{-9}$; odds ratio 1.38). For fixed-threshold LineVul the difference is smaller but still detectable, 1.41% versus 1.06% ($p = 1.9 \times 10^{-2}$; odds ratio 1.34). For near-all-positive calibrated LineVul and calibrated ReGVD, the direction reverses, with non-vulnerable pairs flipping more often; this again shows that label-specific robustness must be interpreted jointly with predicted-positive rate and calibration. Supporting local-family baselines show the same general caution: their average vulnerable and non-vulnerable flip rates are small, but their operating points vary widely.

Table 16

Composition and identifier-renaming ablation diagnostics.

Detector	Max composition flip	Fresh marker	In-vocab	Project style	Worst composition
CodeBERT-Devign-local	0.047	0.025	0.053	0.027	rename + comment
VulBERTa-local	0.063	0.041	0.064	0.041	safe dead code + normalization
LineVul-local	0.047	0.025	0.053	0.027	rename + comment
ReGVD-structural-local	0.041	0.012	0.002	0.010	safe dead code + normalization

Table 17

Label-specific prediction instability.

Detector	Avg. vuln. flip	Avg. non-vuln. flip	Max vuln. flip	Vuln. variant recall
CodeBERT-Devign-local	0.011	0.030	0.016	0.903
VulBERTa-local	0.013	0.025	0.016	0.873
LineVul-local	0.000	0.002	0.000	1.000
ReGVD-structural-local	0.009	0.021	0.017	0.986

This analysis distinguishes benign prediction instability from security-critical instability. In particular, true-positive-to-false-negative flips indicate cases where a detector originally identified a vulnerable function but missed a transformed equivalent. For VulBERTa, this operationalizes to 585 true-positive-to-false-negative changes across 10,040 vulnerable transformed pairs, or 58.3 additional missed detections per 1,000 vulnerable function-variant pairs. Conditional on functions that VulBERTa originally detected as vulnerable, the rate is higher: 109.0 missed detections per 1,000 originally true-positive transformed pairs. These quantities make the 7.01% overall flip rate more concrete: most predictions are stable, but the unstable subset is large enough to matter for CI or code-review gates that repeatedly scan equivalent revisions of vulnerable code. For the exact public rows in Table 18, flip rates compare each transformed variant with its paired original prediction. Differences are vulnerable minus non-vulnerable, and p values use Fisher’s exact test over flip/no-flip counts.

Summary of RQ3

Vulnerable examples are more fragile for the primary VulBERTa exact-public run, where their flip rate exceeds the non-vulnerable flip rate by 2.09 percentage points. However, the direction and magnitude of label-specific fragility depend on detector calibration, so security-critical true-positive-to-false-negative flips should be reported alongside operating point diagnostics.

5.8. RQ4: Mitigation through Variant Aggregation

Answer to RQ4. The mitigation result is primarily negative. Majority vote over original and transformed variants does not materially change CodeXGLUE results because most variants agree with the original prediction. Max-risk aggregation increases recall for CodeBERT-Devign-local, VulBERTa-local, ReGVD-structural-local, and the exact public checkpoints, but it also reduces precision. Calibrated

probability and logit averaging can improve F1 for VulBERTa when the threshold is selected post-hoc on clean CodeXGLUE scores, while disagreement-based abstention improves precision and MCC at the cost of coverage and recall. These are still test-time augmentation diagnostics, not deployable mitigation results. The study therefore identifies a mitigation gap rather than claiming a complete cure: a natural next step is transformation-consistency regularization or certified smoothing during fine-tuning, evaluated jointly on clean F1, TIFR, and VDCP.

We evaluate original-only prediction, majority vote over the original plus variants, max-risk aggregation, probability averaging, calibrated logit averaging, and abstention when variant predictions disagree. The max-risk rule predicts vulnerable if any variant is predicted vulnerable. The calibrated rows use clean-test score thresholds and are therefore diagnostic upper-bound operating points rather than deployable validation-set calibrations. Majority vote is identical to original-only prediction in Table 19 because the original plus eight variant ballots have uniformly high agreement; the row is retained as a baseline check.

Because the most important rows come from exact public checkpoints, we also evaluate aggregation for VulBERTa and LineVul directly. Max-risk aggregation raises VulBERTa recall from 0.535 to 0.645 and F1 from 0.577 to 0.615, at the cost of reducing precision from 0.627 to 0.588. Calibrated probability averaging raises VulBERTa F1 to 0.675 with recall 0.914, but precision drops to 0.535 and the threshold is selected post-hoc. Selective abstention keeps 69.4% of VulBERTa cases and raises precision to 0.670, but recall falls to 0.449. For public LineVul, max-risk raises recall from 0.006 to 0.072 and F1 from 0.013 to 0.126, but the fixed-threshold operating point remains too insensitive for a useful gate.

We also run a small training-time cure probe to test whether transformation-consistency training can improve robustness rather than merely aggregate unstable predictions

Table 18

Label-specific flip-rate tests.

Detector	Vuln. flip	Non-vuln. flip	Difference	Odds ratio	p	Interpretation
VulBERTa-public	0.081 [0.076, 0.087]	0.061 [0.056, 0.065]	0.021	1.38	2.1×10^{-9}	Vulnerable pairs more fragile
LineVul-public	0.014 [0.012, 0.017]	0.011 [0.009, 0.013]	0.004	1.34	1.9×10^{-2}	Small fixed-threshold difference
LineVul-public, calibrated sensitivity	0.001 [0.001, 0.002]	0.003 [0.002, 0.004]	-0.002	0.41	7.3×10^{-3}	Near-all-positive sensitivity
ReGVD-official, validation-calibrated	0.004 [0.003, 0.006]	0.011 [0.010, 0.014]	-0.007	0.35	6.3×10^{-10}	Near-all-positive structural row

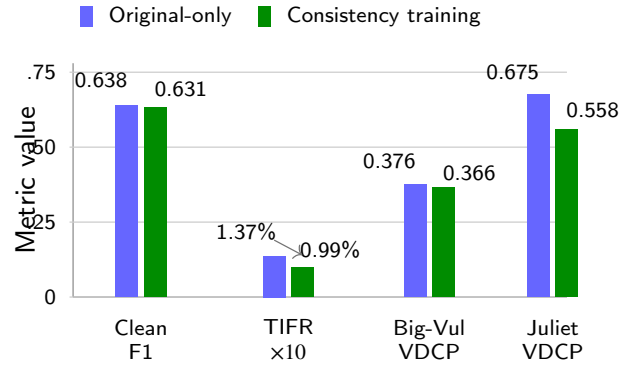
Table 19

Aggregation mitigation results on CodeXGLUE/Devign.

Detector	Strategy	Precision	Recall	F1
CodeBERT-Devign-local	Original only	0.529	0.895	0.665
CodeBERT-Devign-local	Majority vote	0.529	0.895	0.665
CodeBERT-Devign-local	Max-risk	0.513	0.921	0.659
VulBERTa-local	Original only	0.538	0.865	0.663
VulBERTa-local	Majority vote	0.538	0.865	0.663
VulBERTa-local	Max-risk	0.525	0.904	0.665
LineVul-local	Original only	0.459	1.000	0.630
LineVul-local	Majority vote	0.459	1.000	0.630
LineVul-local	Max-risk	0.459	1.000	0.630
ReGVD-structural-local	Original only	0.471	0.984	0.637
ReGVD-structural-local	Majority vote	0.471	0.984	0.637
ReGVD-structural-local	Max-risk	0.460	1.000	0.630

at test time. The probe keeps model capacity fixed: a hashed logistic detector is trained either on original CodeXGLUE training functions or on 6,000 stratified training origins plus their semantics-preserving variants with inherited labels. Table 20 shows that consistency training reduces CodeXGLUE TIFR from 1.37% to 0.99% with a small clean-F1 drop from 0.638 to 0.631. However, VDCP does not improve: Big-Vul VDCP changes from 0.376 to 0.366 and Juliet/SARD VDCP from 0.675 to 0.558. We therefore treat transformation-consistency training as a partial cure for label-preserving score instability, not as a cure for vulnerability-fix ordering. Both rows in Table 20 use the same hashed logistic detector; the consistency row augments 6,000 CodeXGLUE training origins with semantics-preserving variants.

Finally, to understand whether LineVul’s low recall is purely an operating point artifact, we run a post-hoc threshold sensitivity analysis using the clean CodeXGLUE test scores. This is not a replacement for validation-set calibration, but it bounds the effect of the fixed threshold. Lowering the threshold to the post-hoc clean-F1 optimum raises clean recall to 0.998 and clean F1 to 0.630, while the variant F1 becomes 0.630 and the flip rate drops to 0.21%. Thresholds selected to obtain roughly 25%, 50%, and 80%

**Figure 7:** Transformation-consistency training probe.

clean recall produce flip rates of 14.3%, 12.3%, and 7.0%, respectively. Thus, LineVul’s robustness conclusions are highly threshold-dependent; the exact public row remains useful as a documented checkpoint behavior, but calibrated deployment would require a separate validation protocol. The thresholds in Table 22 are diagnostic and are not used for the primary exact-public results.

Table 20

Training-time transformation-consistency cure probe.

Training setting	Train rows	Clean F1	CodeXGLUE TIFR	Big-Vul VDCP	Juliet/SARD VDCP	Interpretation
Original only	21,854	0.638	0.0137	0.376	0.675	Baseline
Transformation-consistency	42,053	0.631	0.0099	0.366	0.558	Partial robustness cure

Table 21

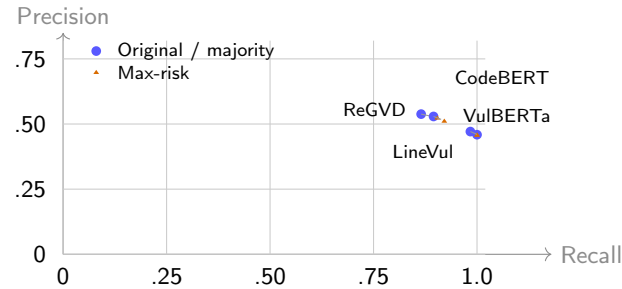
Exact-public aggregation and selective-prediction diagnostics.

Detector	Strategy	Coverage	Precision	Recall	F1
VulBERTa-public	Original fixed	1.000	0.627	0.535	0.577
VulBERTa-public	Max-risk fixed	1.000	0.588	0.645	0.615
VulBERTa-public	Post-hoc mean probability	1.000	0.535	0.914	0.675
VulBERTa-public	Post-hoc mean logit	1.000	0.535	0.908	0.674
VulBERTa-public	Abstain on disagreement	0.694	0.670	0.449	0.538
LineVul-public	Original fixed	1.000	0.615	0.006	0.013
LineVul-public	Max-risk fixed	1.000	0.517	0.072	0.126
LineVul-public	Post-hoc mean probability	1.000	0.460	0.999	0.630
LineVul-public	Abstain on disagreement	0.939	0.667	0.003	0.007

Table 22

Post-hoc LineVul threshold sensitivity.

Setting	Threshold	Clean recall	Clean F1	Variant F1	Flip
Fixed public threshold	0.500000	0.006	0.013	0.037	0.012
Post-hoc clean-F1	0.000012	0.998	0.630	0.630	0.002
Post-hoc recall 0.25	0.000043	0.250	0.331	0.410	0.143
Post-hoc recall 0.50	0.000019	0.501	0.491	0.526	0.123
Post-hoc recall 0.80	0.000014	0.799	0.589	0.600	0.070

**Figure 8:** Precision–recall tradeoff under max-risk aggregation.**Summary of RQ4**

Variant aggregation is useful mainly as a diagnostic. Majority vote changes little, max-risk improves recall at a precision cost, post-hoc averaging shows the upper-bound value of score aggregation, and abstention exposes disagreement. None of these rows should be read as a deployable mitigation without validation-set calibration and explicit precision–recall operating policies.

6. Discussion**6.1. Clean Accuracy Is Not Enough**

Clean benchmark metrics remain important, but they do not capture prediction stability. A detector can achieve strong clean F1 while failing to preserve predictions under harmless edits. Robustness metrics should therefore be reported alongside standard metrics for vulnerability detection.

6.2. Flip Rates Are Not Enough

The study also shows that a single decision-flip rate is not sufficient for comparing detector robustness across operating points. Thresholded flips are deployment-relevant, but they conflate score stability with threshold choice, class prior, and calibration. A near-all-safe or near-all-vulnerable detector can have a low flip rate while still moving examples substantially in score order. The calibration-invariant analysis addresses this gap by integrating over the empirical clean-score distribution and by reporting rank stability. This is why LineVul and validation-calibrated ReGVD are not described as more robust than VulBERTa despite their lower decision-flip rates: their thresholded decisions are stable partly because their operating points are weak, while TIFR exposes larger underlying score movement.

6.3. Two Complementary Metamorphic Relations

The results also distinguish two robustness relations that are easy to merge conceptually but different in practice. The first is label-preserving invariance: a detector should not change its answer when a function is renamed, formatted, commented, normalized, or conservatively rewritten without changing vulnerability status. The second is vulnerability counterfactual dominance: a detector should assign higher

risk to a vulnerable pre-repair function than to its patched counterpart. The Big-Vul patch-pair probe shows that these are not substitutes. A detector can be stable under harmless edits yet still fail to rank a vulnerable function above its repair, or it can move scores under harmless edits while preserving some vulnerable-versus-patched ordering. Separating the two relations is therefore a novelty and measurement issue, not merely an additional experiment.

6.4. Implications

Prediction instability has direct implications for security automation. If a small behavior-preserving edit converts a true positive into a false negative, then an automated gate may allow vulnerable code to pass. Security-sensitive workflows may prefer conservative aggregation strategies that increase recall, even when they introduce additional false positives.

Benchmark maintainers should consider releasing transformed variants, reporting transformation validity, and separating robustness test sets from training data. They should also report token lengths and truncation rates because preprocessing artifacts can affect robustness.

Tool builders can use transformation-based tests as part of pre-deployment validation. Prediction instability across code review revisions can also be monitored as an operational signal. Lightweight aggregation over transformed variants is a baseline mitigation idea, but stronger calibrated and selective aggregation methods are needed before deployment.

6.5. Possible Causes of Brittleness

Detector brittleness may arise from token-level shortcut learning, identifier memorization, sensitivity to formatting, truncation artifacts, or weak modeling of data and control dependencies. The relative importance of these causes may vary by detector family and transformation type.

To move beyond marginal flip counts, we fit a diagnostic logistic regression over the exact public CodeXGLUE pairs, predicting whether a transformed variant flips relative to its original. The model includes transformation family, vulnerability label, original score margin, code length, a token-length proxy, and a token-length truncation proxy. For VulBERTa, identifier renaming has the largest transformation association (odds ratio 223.3 relative to the layout-only reference), followed by safe dead-code carriers (187.7), comment insertion (184.3), dead-branch insertion (168.5), and code normalization (21.2). Larger original score margins reduce flip odds (OR 0.39), consistent with threshold-adjacent examples being more brittle. The token-length truncation proxy is not significant for VulBERTa (OR 1.04, $p = 0.156$), but is significant for LineVul (OR 1.33, $p < 10^{-12}$). These associations are not causal explanations of internal model behavior. They show that flip probability is statistically associated with transformation family and calibration margin more strongly than with length alone, but they do not identify the internal mechanism by which a detector changes its prediction.

Our experiments identify where predictions change, but they do not fully explain why each model changes internally.

We do not perform token-level attribution, representation probing, or a systematic ablation over transformation strength for the exact public checkpoints. The 512-origin composition and identifier-ablation probe partially addresses interaction effects, but it uses local detector-family baselines and therefore remains supporting evidence rather than a replacement for full exact-checkpoint composition runs.

7. Threats to Validity

7.1. Construct Validity

The main construct validity threat is that a transformation may not preserve semantics in all contexts. We mitigate this risk through conservative transformation rules, syntax checks, deterministic pre-audit, and a completed two-annotator human annotation protocol. The human audit identified no invalid cases but flagged 11 conservative uncertain cases; these were excluded from our analysis to ensure high-fidelity validity claims. The revised validity reporting also avoids relying only on a pooled invalidity bound: per-transformation bounds are 2.5% for transformations with 120 audited examples and 12.5% for the rare control-flow rewrite. A small compile-anchor check over stratified audit rows showed that almost no snippets compile as standalone translation units without project headers, types, and macros; the single original/variant pair that compiled did so on both sides. This confirms that compilation is useful as an anchor for the few standalone cases but cannot replace transformation preconditions, parse screening, and human audit for benchmark snippets. The audit validates the transformation relation, not the correctness of the underlying benchmark vulnerability labels. Vulnerability datasets can contain noisy labels, duplicate or near-duplicate functions, project-specific leakage, and incomplete CWE annotations. Because our main metrics are paired within each original function, label noise does not by itself create prediction flips, but it can affect whether a flip is classified as harmful, beneficial, or security-critical.

7.2. Internal Validity

Prediction changes may be caused by tokenization, truncation, preprocessing, or thresholding rather than model reasoning. We mitigate this threat by using fixed thresholds, logging token lengths, reporting truncation rates, and applying transformations only at evaluation time. The calibration-invariant TIFR and rank-stability analysis further mitigates threshold confounding by separating score-order movement from a single binary operating point, although it still does not reveal the internal causal mechanism of a detector.

7.3. External Validity

Function-level datasets may not fully represent whole-project vulnerability detection or deployment in industrial CI/CD systems. We mitigate this threat by using Big-Vul and DiverseVul as EVPs and by limiting claims to function-level vulnerability detection. The strongest detector-specific conclusions are restricted to CodeXGLUE/Devign, where

exact public VulBERTa and LineVul checkpoints are executed on the full transformed test set. Big-Vul and DiverseVul support narrower claims about transformation applicability and imbalance-aware metric behavior; they are not evidence for full cross-dataset detector rankings. The CodeXGLUE/DeVign split used here does not provide sufficiently reliable fine-grained CWE labels for a primary per-CWE robustness analysis. Big-Vul and DiverseVul contain CWE metadata, but in this article they are sampled EVPs rather than full exact-checkpoint evaluation datasets. The counterfactual patch-pair probe uses the 101 matched Big-Vul before/after pairs available in the sample and reports exact-public CWE-stratified diagnostics only for cells with at least five pairs. Consequently, vulnerability-type-conditioned robustness remains a supporting diagnostic rather than a central claim of this study. The sparse CWE-conditioned exact-checkpoint rows are a limitation of the current artifact rather than evidence that all vulnerability classes are equally robust. A future extension should preserve CWE metadata through variant generation and compute per-CWE flip rates and patch-pair ordering only for vulnerability classes with enough positives to support meaningful intervals.

Detector coverage is also limited. The strongest evidence uses full-test public VulBERTa and LineVul checkpoints, the official ReGVD graph implementation on the full transformed CodeXGLUE test set, and a GraphCodeBERT-DeVign checkpoint on a 512-origin subset. ReGVD's non-collapsed row depends on validation-threshold calibration; at the upstream fixed 0.5 threshold it collapses to all-safe predictions. The AST-structural-RF and AST-structural-ExtraTrees rows are working non-collapsed local structural baselines, not public DeVign/ReVeal checkpoints. The ExtraTrees variant is evaluated on the full transformed CodeXGLUE test set with clean F1 0.636, flip rate 1.67%, and TIFR 3.20%. After discounting the degenerate LineVul fixed-threshold row, the clearest full-test exact-checkpoint robustness signal still comes from VulBERTa, while calibrated ReGVD provides public structural implementation coverage. Modern graph detectors with stronger public checkpoints, UniXcoder vulnerability classifiers, proprietary scanners, and LLM-based zero-shot detectors may behave differently. The contribution is therefore best read as a reusable robustness benchmarking protocol with full-test CodeXGLUE evidence for selected public detectors, not as a comprehensive ranking of vulnerability detectors or a claim about AI-assisted security tools in general.

Some transformations are intentionally conservative but synthetic. For example, `if (0)` dead-code carriers are useful for testing whether a model reacts to unreachable code, yet developers may more often introduce logging, feature-flagged, or preprocessor-guarded code. We therefore interpret dead-code results as stress tests for superficial token reliance rather than frequency estimates of ordinary maintenance edits. We also do not mine CodeXGLUE project histories to estimate how often each transformation appears in real commits. The motivation is grounded in prior refactoring and repository-mining studies, but the benchmark itself should be read as controlled robustness infrastructure rather

than a frequency-weighted model of field edits. Similarly, the current identifier-renaming intervention uses one fresh marker style in the full public-checkpoint experiment. The 512-origin ablation shows that in-vocabulary and project-style names can also produce flips for local detector-family baselines, but it does not prove the same mechanism for the public checkpoints. Stronger future transformations should include dependency-checked statement reordering, idiom-preserving loop rewrites, and type-preserving expression refactors. A further extension is to scale the vulnerability-semantics-aware counterfactual patch-pair probe beyond the 101 Big-Vul pairs used here, for example with Juliet/SARD, and report CWE-conditioned intervals only where the patch-pair count is sufficient.

7.4. Conclusion Validity

Observed robustness drops may depend on sample selection or detector configuration. We mitigate this threat by using paired analyses, confidence intervals, multiple detector families, and a reproducible artifact package. For LineVul in particular, the fixed 0.5 function-level threshold may not be the operating point a practitioner would choose after calibration. Alternative thresholds could improve recall and change flip-rate magnitudes, although the reported row remains the exact public checkpoint behavior under the documented runner. The central full-test robustness signal is also detector-limited. After discounting LineVul's weak function-level operating point and ReGVD's near-all-positive calibrated row, VulBERTa is the only full-test public checkpoint that yields an interpretable non-degenerate robustness signal. The article therefore supports a scoped claim about the evaluated public detectors and a reusable benchmark protocol, not a universal claim about all learning-based or AI-assisted vulnerability detectors.

8. Conclusion

This article argues that learning-based vulnerability detectors should be evaluated not only by clean benchmark performance, but also by robustness under semantics-preserving code transformations. Clean F1, PR-AUC, balanced accuracy, and MCC remain important measures of predictive quality, yet they do not reveal whether a detector preserves its behavior when code is reformatted, renamed, commented, normalized, or rewritten in conservative behavior-preserving ways. For software engineering use cases such as code review, continuous integration, and security regression testing, this stability is a first-order software-quality property.

We presented a low-compute empirical benchmark for transformation-based metamorphic testing of vulnerability detection tools. The benchmark keeps each variant paired with its original function, applies detector inference without retraining, and reports prediction flip rates, confidence intervals, paired tests, transformed performance, and label-specific fragility. It also adds calibration-invariant score metrics, including threshold-integrated flip rate and rank-displacement robustness, to separate score movement from fixed-threshold artifacts. Finally, the benchmark includes an exploratory

VDCP patch-pair probe that distinguishes stability under harmless edits from the ability to rank vulnerable pre-repair code above patched code. The artifact contains transformed dataset samples, public and supporting prediction files, composition and identifier-ablation diagnostics, aggregation and mechanism analyses, patch-pair results, a consistency-training probe, and audit materials.

The results provide a scoped warning against inferring deployment robustness from clean benchmark performance alone. On the full transformed CodeXGLUE test set, exact public VulBERTa exhibits a measurable 7.01% prediction flip rate, a clean-to-variant F1 drop from 0.577 to 0.555, and 58.3 additional true-positive-to-false-negative changes per 1,000 vulnerable transformed pairs. Exact public LineVul has a lower flip rate of 1.22%, but this apparent stability is limited by near-zero vulnerable recall. Official ReGVD also illustrates the importance of calibration: its fixed threshold collapses, whereas validation-threshold calibration produces a non-collapsed full-test structural row with a 0.81% flip rate but weak balanced accuracy and a 7.23% TIFR. LineVul similarly has only 1.22% decision flips but 9.76% TIFR, showing that low binary flip rates can mask substantial score-order instability. Across transformation families, identifier renaming, comment insertion, dead-branch insertion, and safe dead-code carriers are more disruptive than layout-only changes. The exploratory patch-pair probe adds a vulnerability-specific caution: exact public VulBERTa, LineVul, and validation-calibrated ReGVD achieve only 30.7–45.5% VDCP on original Big-Vul before/after fix pairs, and thresholded decision-pair success is at most 4.0%. On 400 Juliet/SARD construction-guaranteed C bad/good pairs, the same public detectors reach only 33.0–46.3% VDCP, with decision-pair success at most 3.7%. Thus a detector's robustness claim should not rest only on whether it preserves predictions under harmless edits; it should also be tested against vulnerability-removing counterfactuals where suitable patch pairs are available.

The practical implication is that robustness evaluation should become part of the standard reporting and quality-assurance package for vulnerability detectors. Researchers can use transformation tests to expose sensitivity to superficial source features; benchmark designers can report transformed variants alongside clean test sets; and tool builders can use paired robustness tests as pre-deployment checks for learning-based security tools in code-review and CI pipelines. Simple aggregation over transformed variants is not a complete mitigation: majority vote changes little, max-risk aggregation trades precision for recall, and post-hoc calibrated averaging is diagnostic rather than deployable. A low-compute transformation-consistency training probe reduces CodeXGLUE TIFR from 1.37% to 0.99% with a small clean-F1 drop, but it does not improve Big-Vul or Juliet/SARD VDCP. Practical deployment therefore still requires validation-set calibration, better model design, stronger training-time robustness objectives, and semantic reasoning.

Overall, the strongest empirical claim is deliberately scoped to CodeXGLUE/DeVign, where exact public checkpoints are evaluated on the full transformed test set and the semantic audit validates a randomly sampled subset with strong annotator agreement, no invalid examples, a 95% upper bound of 0.43% on the invalid rate, and conservative exclusions. Big-Vul and DiverseVul strengthen external validity only in a limited sense: they show that the transformations can be applied to additional vulnerability corpora and demonstrate the need for imbalance-aware metrics. The 512-origin composition probe suggests that composed edits can still produce measurable flips, but the full-test exact-checkpoint claims remain single-transformation claims. This distinction keeps the conclusions useful while avoiding overstatement from sampled, supporting baselines.

Acknowledgement

We acknowledge the time and facilities support provided by Ho Chi Minh City University of Technology (HCMUT), VNU-HCM for this study.

Data and Code Availability

The source code, experiment scripts, manuscript sources, and replication materials are available in the GitHub repository <https://github.com/LeoTTTPhat/vd-robustness>.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this article.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

- [1] Y. Zhou, S. Liu, J. Siow, X. Du, Y. Liu, DeVign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, in: *Advances in Neural Information Processing Systems*, 2019, pp. 10197–10207.
- [2] M. Fu, C. Tantithamthavorn, LineVul: A transformer-based line-level vulnerability prediction, in: *Proceedings of the International Conference on Mining Software Repositories*, 2022.
- [3] H. Hanif, S. Maffeis, VulBERTa: Simplified source code pre-training for vulnerability detection, in: *International Joint Conference on Neural Networks*, 2022.
- [4] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. B. Clement, D. Drain, D. Jiang, D. Tang,

- G. Li, L. Zhou, L. Shou, L. Zhou, M. Tufano, M. Gong, M. Zhou, N. Duan, N. Sundaresan, S. K. Deng, S. Fu, S. Liu, CodeXGLUE: A machine learning benchmark dataset for code understanding and generation, arXiv preprint arXiv:2102.04664 (2021).
- [5] J. Fan, Y. Li, S. Wang, T. N. Nguyen, A c/c++ code vulnerability dataset with code changes and cve summaries, in: Proceedings of the International Conference on Mining Software Repositories, 2020.
- [6] Y. Chen, Z. Ding, L. Alowain, X. Chen, D. Wagner, DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection, in: Proceedings of the International Symposium on Research in Attacks, Intrusions and Defenses, 2023.
- [7] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, M. Zhou, CodeBERT: A pre-trained model for programming and natural languages, in: Findings of the Association for Computational Linguistics: EMNLP, 2020.
- [8] D. Q. Nguyen, K. Rieck, L. Grunske, C. Kästner, Revisiting graph neural networks for vulnerability detection, in: Proceedings of the International Conference on Software Engineering, 2022.
- [9] E. Murphy-Hill, C. Parnin, A. P. Black, How we refactor, and how we know it, IEEE Transactions on Software Engineering 38 (1) (2012) 5–18. doi:10.1109/TSE.2011.41.
- [10] M. Kim, T. Zimmermann, N. Nagappan, An empirical study of refactoring challenges and benefits at microsoft, IEEE Transactions on Software Engineering 40 (7) (2014) 633–649. doi:10.1109/TSE.2014.2318734.
- [11] A. Peruma, S. Simmons, E. A. AlOmar, C. D. Newman, M. W. Mkaouer, A. Ouni, An empirical investigation of how and why developers rename identifiers, in: Proceedings of the International Workshop on Refactoring, 2018, pp. 26–33.
- [12] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, Z. Q. Zhou, Metamorphic testing: A review of challenges and opportunities, ACM Computing Surveys 51 (1) (2018) 1–27. doi:10.1145/3143561.
- [13] N. Yefet, U. Alon, E. Yahav, Adversarial examples for models of code, Proceedings of the ACM on Programming Languages 4 (OOPSLA) (2020) 162:1–162:30. doi:10.1145/3428230.
- [14] H. Zhang, Z. Li, G. Li, L. Ma, Y. Liu, Z. Jin, Generating adversarial examples for holding robustness of source code processing models, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 34, 2020, pp. 1169–1176. doi:10.1609/aaai.v34i01.5469.
- [15] P. Bielik, M. Vechev, Adversarial robustness for code, in: Proceedings of the 37th International Conference on Machine Learning, Vol. 119 of Proceedings of Machine Learning Research, 2020, pp. 896–907.
- [16] M. R. I. Rabin, N. D. Q. Bui, K. Wang, Y. Yu, L. Jiang, M. A. Alipour, On the generalizability of neural program models with respect to semantic-preserving program transformations, Information and Software Technology 135 (2021) 106552. doi:10.1016/j.infsof.2021.106552.
- [17] J. Henkel, G. Ramakrishnan, Z. Wang, A. Albarghouthi, S. Jha, T. Reps, Semantic robustness of models of source code, in: Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering, 2022.
- [18] Z. Yang, J. Shi, J. He, D. Lo, Natural attack for pre-trained models of code, in: Proceedings of the International Conference on Software Engineering, 2022, pp. 1482–1493. doi:10.1145/3510003.3510146.
- [19] H. Zhang, Z. Li, G. Li, L. Ma, Y. Liu, Z. Jin, Towards robustness of deep program processing models: Detection, estimation, and enhancement, ACM Transactions on Software Engineering and Methodology (2022).
- [20] S. Wang, Z. Li, H. Qian, C. Yang, Z. Wang, M. Shang, V. Kumar, S. Tan, B. Ray, P. Bhatia, R. Nallapati, M. K. Ramanathan, D. Roth, B. Xiang, ReCode: Robustness evaluation of code generation models, in: Proceedings of the Annual Meeting of the Association for Computational Linguistics, 2023.
- [21] Y. Li, S. Qi, C. Gao, Y. Peng, D. Lo, D. Y.-A. Lo, M. R. Lyu, Z. Xu, Understanding the robustness of transformer-based code intelligence via code transformation: Challenges and opportunities, IEEE Transactions on Software Engineering 51 (2) (2025) 521–547.
- [22] S. Chakraborty, R. Krishna, Y. Ding, B. Ray, Deep learning based vulnerability detection: Are we there yet?, IEEE Transactions on Software Engineering (2022).
- [23] K. Pei, Y. Cao, J. Yang, S. Jana, DeepXplore: Automated whitebox testing of deep learning systems, in: Proceedings of the ACM Symposium on Operating Systems Principles, 2017, pp. 1–18. doi:10.1145/3132747.3132785.
- [24] Y. Tian, K. Pei, S. Jana, B. Ray, DeepTest: Automated testing of deep-neural-network-driven autonomous cars, in: Proceedings of the International Conference on Software Engineering, 2018, pp. 303–314. doi:10.1145/3180155.3180220.
- [25] L. Ma, F. Juefei-Xu, F. Zhang, J. Sun, M. Xue, B. Li, C. Chen, T. Su, L. Li, Y. Liu, J. Zhao, Y. Wang, DeepGauge: Multi-granularity testing criteria for deep learning systems, in: Proceedings of the IEEE/ACM

International Conference on Automated Software Engineering, 2018, pp. 120–131. doi:10.1145/3238147.3238202.

- [26] J. M. Zhang, M. Harman, L. Ma, Y. Liu, Machine learning testing: Survey, landscapes and horizons, *IEEE Transactions on Software Engineering* 48 (1) (2022) 1–36. doi:10.1109/TSE.2019.2962027.
- [27] L. Sun, A. Oprea, E. Wong, Semantics-preserving evasion of llm vulnerability detectors, *arXiv preprint arXiv:2602.00305* (2026). arXiv:2602.00305.
- [28] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, S. K. Deng, C. B. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang, M. Zhou, GraphCodeBERT: Pre-training code representations with data flow, in: *International Conference on Learning Representations*, 2021.
- [29] NSA Center for Assured Software, NIST SARD: Juliet test suite for C/C++, version 1.3 (2017). doi:10.5281/zenodo.4701387.
URL <https://doi.org/10.5281/zenodo.4701387>